PSEUDONYM RESOLUTION AND FORMAL VERIFICATION OF AN UNLINKABLE TWO-MESSAGE IOT AUTHENTICATION PROTOCOL

ON

Submitted in partial fulfillment of the requirements of
the degree of

Bachelor of Engineering

(Information Technology)

By

Jay Kerkar – Roll No (16)
Chaitanya Kelkar – Roll No (15)
Chirag Poornamath – Roll No (26)
Yash Kolekar – Roll No (17)

Under the guidance of

DR. SHANTA SONDUR

Department of Information Technology
VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY,
Chembur, Mumbai 400074
(An Autonomous Institute, Affiliated to University of Mumbai)

AY (2026-27)

Vivekanand Education Society's
Institute of Technology

(Autonomous Institute Affiliated to University of Mumbai, Approved by AICTE & Recognised by Govt. of Maharashtra)
NAAC accredited with 'A' grade

Since 1962

Certificate

This is to certify that project entitled

"Pseudonym Resolution and Formal Verification of an Unlinkable Two-Message IoT Authentication Protocol"

Group Members Names

[FLAG F2] Jay Kerkar (Roll No. 16)
[FLAG F2] Chaitanya Kelkar (Roll No. 15)
[FLAG F2] Chirag Poornamath (Roll No. 26)
[FLAG F2] Yash Kolekar (Roll No. 17)

In partial fulfilment of the Fourth Year (Sem. VI-Sem.VII), Bachelor of Engineering (B.E.) Degree in Information Technology at the University of Mumbai

Academic Year 2026-27

Dr. Shanta Sondur
Project Mentor

External Examiner

Dr.(Mrs.)Shalu Chopra
H.O.D

Dr.(Mrs.)J.M. Nair
Principal

Date: / /2026
Place:
VESIT, Chembur

College Seal

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

_____

(Signature)

Jay Kerkar (16)

_____

(Signature)

Chaitanya Kelkar (15)

_____

(Signature)

Chirag Poornamath (26)

_____

(Signature)

Yash Kolekar (17)

Abstract

Lightweight authentication protocols for the Internet of Things (IoT) often hide a device's identity behind a pseudonym that changes with every epoch, so that an eavesdropper cannot link two sessions of the same device. In the two-message protocol of Al-Rasheed et al. (IEEE Transactions on Consumer Electronics, 2026), the device sends the pseudonym in clear and encrypts everything else under a key that is specific to the device. The server therefore has to find out which device sent a message before it can decrypt or verify it. This project shows, from the specification alone, that every correct server needs this *pseudonym resolution* step, and that the protocol neither specifies it nor includes it in its cost model. The protocol was implemented in Python, with a C port of the device side for the ESP8266, and both were checked byte for byte against independent reference vectors (15 checks each). Seven resolution strategies were added and their probe counts were measured and checked against closed forms: four that keep the wire format, and three adapted from RFID privacy research (randomized hash locks, key trees and rainbow time–memory trade-offs). The counts were priced with the base paper's own per-operation constants in an OMNeT++ model of its stated network (960 runs). At one device every strategy keeps the published 23.68 ms per session. At 1,000 devices with 5% adversaries, a precomputed table keeps a session at 24.4 ms, while the on-demand counter scan reaches about 44 s, because each forged pseudonym forces a full scan that the block-list never sees. Counter epochs also lock a device out permanently after eight dropped messages. A ProVerif analysis shows that session-key secrecy and agreement hold for every correct resolver; unlinkability holds with counter epochs and fails with clock epochs; and injective device authentication needs replay state that the specification leaves open. The code, models and data are released openly.

Keywords–*IoT authentication, pseudonym resolution, unlinkability, private identification, scalability, OMNeT++, formal verification, ProVerif*

Contents

Chapter 1. Introduction **2**
1.1 Introduction . . . 2
1.2 Problem Definition . . . 3
1.3 Objectives . . . 3
1.4 Motivation . . . 4
1.5 Scope of the Work . . . 4
1.6 Feasibility Study . . . 5
1.7 Organization of the report . . . 6

Chapter 2. Literature Survey **8**
2.1 Introduction . . . 8
2.2 Review of Literature Survey . . . 8
2.3 Research gaps . . . 14
2.4 Conclusion . . . 15

Chapter 3. Proposed Methodology **17**
3.1 Introduction . . . 17
3.2 Requirement Gathering . . . 17
3.2.1 Functional Requirements . . . 18
3.2.2 Non Functional Requirements . . . 19
3.2.3 User Requirements . . . 19
3.3 Proposed Algorithms . . . 20
3.4 Gantt chart for Timeline . . . 24
3.5 Architectural Diagram . . . 24
3.5.1 UML Diagrams . . . 25
3.5.2 Block Diagram . . . 27
3.5.3 Data Flow Diagram . . . 28
3.5.4 Timeline Chart . . . 29
3.6 Hardware Requirements . . . 30
3.7 Software Requirements . . . 31
3.8 Cost Estimation . . . 32
3.9 Conclusion . . . 32

Chapter 4. Implementation & Results **35**
4.1 Introduction 35
4.2 Implementation 35
4.3 Results of Implementation 38
4.4 Result Analysis 43
4.5 Limitations 45
4.6 Observation/Remarks 46
4.7 Conclusion 46

Chapter 5. Conclusion **49**
5.1 Limitations 49
5.2 Conclusion 49
5.3 Future Scope 50

References **52**

Appendix **54**
A.I. Publication Details 54
A.II. IPR: Copyright/Patent/etc. 54

List of Figures

3.1 Where pseudonym resolution sits in the exchange. The server needs $k_V^\star$ to open M_1, but V is hidden behind *PID*. 21
3.2 Gantt chart of the project timeline (from repository commit dates) 24
3.3 Layered architecture of the implementation and evaluation 25
3.4 Use-case diagram 25
3.5 Class diagram of the Python implementation 26
3.6 Sequence diagram of one authentication 27
3.7 Block diagram of the system 28
3.8 Data flow diagram, level 0 (context) 28
3.9 Data flow diagram, level 1 29
3.10 Timeline chart of project milestones (2026) 29

4.1 Probes per message at one server, measured on the implementation (log–log): (a) honest authentication, (b) a forged, random pseudonym 41
4.2 Mean session latency against network size in the OMNeT++ model (log scale; band: minimum to maximum over five seeds). Dotted: the paper's cost model with no resolution. Dashed: the paper's 23.68 ms. (a) No adversaries, (b) 5% adversaries 42
4.3 (a) Server storage at one server (Table VI's 320 bits per device plus the resolution state); (b) resolution time per authentication ($T_H = 0.32$ ms per probe) against storage at $N = 1000$ 42

List of Tables

2.1 Comparative Summary of Literature Survey 13
2.2 Research Gap Analysis 14

3.1 Notation 20
3.2 Design space of resolution strategies. One table entry is charged at 40 B; "probes" are expected H_2 evaluations per message in steady state. 23
3.3 Hardware requirements 30
3.4 Software requirements 31
3.5 Cost estimation 32

4.1 Test suite (re-run on 3 Nov 2026: 125 passed) 39
4.2 Measured H_2 probes per message at one server with n enrolled devices (`measured.csv`) 41
4.3 Strategies ranked by mean session latency at $N = 1000$ (5% adversaries); mean of five seeds 42
4.4 Next three honest handshakes after k dropped M_1 messages (counter epochs, $F = 8$) 43
4.5 ProVerif results 43
4.6 Objectives and the evidence obtained 45

ACKNOWLEDGEMENT

The project report on “Pseudonym Resolution and Formal Verification of an Unlinkable Two-Message IoT Authentication Protocol” is the outcome of the guidance, moral support and devotion bestowed on our group throughout our work. For this we acknowledge and express our profound sense of gratitude to everybody who has been the source of inspiration throughout project preparation. First and foremost we offer our sincere thanks to Dr.(Mrs.) Shalu Chopra, Head of the Department of Information Technology, to **[FLAG F3]**, Deputy Head of the Department, and to our project guide Dr. Shanta Sondur, **[FLAG F4]**, for their valuable inputs and the consistent guidance and support they provided. We also thank Vivekanand Education Society’s Institute of Technology for providing a stimulating atmosphere and a conducive work environment.

(Signature)
Jay Kerkar (16)

(Signature)
Chaitanya Kelkar (15)

(Signature)
Chirag Poornamath (26)

(Signature)
Yash Kolekar (17)

CHAPTER: 1

INTRODUCTION

Chapter 1

Introduction

1.1 Introduction

The Internet of Things (IoT) connects large numbers of small, resource-constrained devices, such as sensors, meters and controllers, to servers that collect their data and issue commands. Before a device and a server exchange application data they must authenticate each other and agree on a fresh session key. Because the devices have little processing power, memory and energy, much recent work proposes *lightweight* mutual-authentication protocols that use only symmetric primitives (hash functions, message authentication codes and authenticated encryption) during the online phase, and reserve public-key operations, if any, for an offline set-up phase.

Privacy is a second requirement. If a device sends the same identifier in every session, an eavesdropper can follow it across sessions and locations. Many protocols therefore replace the identity with a *pseudonym* that changes over time. If two pseudonyms of the same device cannot be linked without a secret, the protocol is said to provide *unlinkability*.

Al-Rasheed et al. [1] propose such a protocol, a low-cost hybrid elliptic-curve-cryptography (ECC) authentication protocol published in the *IEEE Transactions on Consumer Electronics* in 2026. A trusted authority (TA) gives every device–server pair a pairwise key in an offline phase. Online, the device and the server authenticate in two messages using symmetric primitives only. The device sends a pseudonym $PID = H_2(\lambda_V \| t)$ in clear, where λ_V is the device's long-term secret and t is an epoch. The epoch t and the device's nonce N_i travel inside a ciphertext encrypted under the pairwise key $k^\star$.

This project takes that protocol as its specification. It implements it, verifies the implementation against independent test vectors, and then studies one step that the specification needs but does not describe: how the server finds out which device a pseudonym belongs to. We call this step *pseudonym resolution*. The project measures what resolution costs, how that cost depends on the way the open choices are filled, what failure modes each choice introduces, and which security properties depend on the choice.

1.2 Problem Definition

In the protocol of [1], the server cannot decrypt the first message M_1 until it knows which device sent it, because the key $k^\star$ is specific to the device. Yet the pseudonym in M_1 is designed to hide exactly that. Every correct server must therefore map the pseudonym *PID* to a device *V* and an epoch *t* by computing pseudonyms from the secrets it holds, before any cryptographic check can be made.

The specification says only that the pseudonym is "rotated with time (or epoch) *t*". It does not say whether *t* is a per-device counter or a clock, nor whether the server searches on demand or prepares a table in advance. Its cost model (Table V and Eq. 22 of [1]) charges the server a constant $2T_H + 2T_{SE/D}$ per session whatever the number of devices, so it contains no resolution term at all. RFID privacy research, which has studied the same problem under the name *private identification* for about twenty years [2, 3, 4, 5, 6, 7], shows that without precomputation a server that must identify one of *N* devices from an unlinkable response does $O(N)$ work.

The problem addressed by this project is therefore:

> *To determine, for the two-message protocol of [1], how the unspecified pseudonym-resolution step can be filled, what each choice costs in time and memory as the network grows, how each choice behaves under forged messages and dropped messages, and which of the protocol's security properties depend on the choice.*

The problem matters to anyone who would deploy or extend the protocol: a server operator who implements resolution naively may see latency grow from milliseconds to tens of seconds, or may expose devices to a permanent lockout, without any warning from the published cost model.

1.3 Objectives

The objectives of the project are:

1. To implement the online exchange of Section III of [1] in Python, and the device side in C for the ESP8266, and to show that both reproduce independently generated reference vectors byte for byte (15 checks each).
2. To show from the specification alone that every correct server must perform pseudonym resolution, and to identify the choices that the specification leaves open.
3. To implement seven resolution strategies (four that keep the wire format and three

adapted from RFID research), and to measure their probe counts for $N = 1$ to 10,000 devices, checking them against closed-form expressions.

4. To price the measured probe counts with the paper's own per-operation constants in an OMNeT++ model of the paper's stated network, for networks of 1 to 1,000 devices with and without adversaries (960 simulation runs), and to rank the strategies by session latency and server storage.
5. To characterise the failure modes of each strategy, namely denial of service through forged pseudonyms and desynchronisation through dropped messages.
6. To verify with ProVerif 2.05 which security properties (session-key secrecy, agreement, injective authentication, forward secrecy and unlinkability) hold independently of the resolver and which depend on the epoch and replay choices.
7. To release the implementation, scripts, models and data openly so that every reported number can be reproduced.

1.4 Motivation

Published lightweight protocols are usually judged by the cost figures in their own evaluation tables, which count hash and encryption operations per session. If a step that grows with the size of the network is missing from those tables, the published comparison favours the protocol for reasons that will not survive deployment. The base protocol of [1] claims a per-session cost of 23.68 ms, but that figure is only meaningful once the server knows which device it is talking to.

The project was motivated by a concrete observation made while implementing the protocol: our first server implementation had to scan every enrolled device over a window of epochs for every incoming message, and this cost did not appear anywhere in the paper. RFID research offers several well-studied answers to the same problem, such as hash locks, hash chains, key trees, time–memory trade-offs and time-based tables, but they had not been compared under the cost constants and network conditions of this protocol. Doing so makes the protocol's cost claim testable, gives implementers a concrete recommendation, and shows which parts of the specification need to be stated explicitly.

1.5 Scope of the Work

In scope.

- The online two-message exchange of Section III (Eqs. 8–24) of [1], instantiated with AES-256-GCM, HKDF-SHA256 and SHA-256, as the paper's instantiation notes suggest.

- A Python reference implementation (trusted authority, device, server, session log, demonstration command-line tools) and a C port of the device side, with independent reference vectors.
- Seven pseudonym-resolution strategies (O1–O4, TM, CR, MW), their probe counts, server state and closed forms.
- An OMNeT++ model of the paper's Table II network in which only the resolution term is added to the paper's cost model.
- Denial-of-service and desynchronisation behaviour, and symbolic security analysis in ProVerif.

Intended users. Researchers and engineers who design, evaluate or deploy pseudonym-based IoT authentication, and reviewers who want to reproduce the results.

Out of scope.

- Re-evaluating the other results of [1]; the protocol is taken as the specification.
- Device mobility and hand-off; devices are static and coverage is an abstract predicate.
- Re-keying (Eq. 24), which remains a stub.
- Hardware timings: probes are priced with the paper's constant $T_H = 0.32\,\text{ms}$.
- Side channels (timing, power, fault injection), computational security proofs, and production-grade key storage.

1.6 Feasibility Study

Technical feasibility. All components use mature, freely available tools: Python with the `cryptography` library, BearSSL for the C port, OMNeT++ 6.4 for network simulation and ProVerif 2.05 for symbolic verification. The whole evaluation runs on an ordinary laptop: the full probe-count sweep takes about three minutes, the 960-run OMNeT++ sweep about ten seconds in the original environment, and every ProVerif model finishes in under one second. The device port runs on a low-cost ESP8266 (NodeMCU) board, and can be checked without hardware through a host build.

Operational feasibility. The implementation provides command-line tools for enrolment (`hea-ta`), the server (`hea-server`) and the device (`hea-device`), so the protocol can be demonstrated on one machine in three terminals. A continuous-integration workflow runs the test suite (with a 90% coverage gate) and the C vector check on every push.

Economic feasibility. All software used is free or open source, and the computers used were already available to the team. The only hardware specific to the project is the ESP8266 development board used for the optional on-device demonstration (Sec-

tion 3.8).

Schedule feasibility. The work was organised in phases (specification and implementation, conformance, resolution study, simulation, verification, reporting) that fit within the academic year; the schedule is shown in Section 3.4.

1.7 Organization of the report

- **Chapter 1** introduces the problem domain, defines the problem, and states the objectives, motivation, scope and feasibility of the project.
- **Chapter 2** reviews the base protocol and the RFID private-identification literature, compares them, and identifies the research gaps that the project addresses.
- **Chapter 3** presents the proposed methodology: requirements, the protocol and the seven resolution strategies, the project timeline, the architecture and UML diagrams, and the hardware, software and cost requirements.
- **Chapter 4** describes the implementation, presents the results (conformance, probe counts, network simulation, desynchronisation and formal verification), and analyses them.
- **Chapter 5** states the limitations, concludes the report and outlines future work.

The report ends with the references and an appendix containing the publication details.

CHAPTER: 2

LITERATURE SURVEY

Chapter 2

Literature Survey

2.1 Introduction

The project sits at the meeting point of two bodies of work: lightweight mutual authentication for IoT devices, represented here by the protocol of Al-Rasheed et al. [1] that serves as our specification, and *private identification* in RFID systems, where a reader must identify a tag from a response that is deliberately unlinkable. Studying both is necessary before building anything, because the problem that the project addresses, a server that must identify a device from an unlinkable pseudonym, is new only to the first body of work. The second has studied it since the early 2000s and offers a range of solutions with known time and memory costs.

The survey therefore reviews the base protocol in detail, then the main RFID approaches to private identification: randomized hash locks, hash chains, key trees, time–memory trade-offs and time-based tables, together with the comparative study that analysed them. The sources are peer-reviewed journal and conference papers. For each one we record the problem, the method, the cost of identification, its limitations, and what the project takes from it. The comparison in Table 2.1 and the gap analysis in Section 2.3 then show why the project is needed.

2.2 Review of Literature Survey

Paper 1: A Low-Cost Hybrid Elliptic Curve Cryptography Authentication Protocol for Trustworthy Internet of Things Communication

Authors and year: A. Al-Rasheed, R. Khan, A. Ahmad, A. Saeed, F. Alturise and S. Alkhalaf, 2026 [1]. *IEEE Transactions on Consumer Electronics*, vol. 72, no. 2, pp. 4483–4491. Link: `https://doi.org/10.1109/TCE.2026.3662053`.

Summary. The paper proposes a mutual-authentication protocol between IoT devices and servers that confines expensive operations to an offline phase. A trusted authority derives each entity's long-term secret $\lambda_X = H_1(ID_X \| s)$ and each device–server pair's pairwise key $k^\star = \mathrm{KDF}(\lambda_V \| \lambda_R)$ (Eqs. 8–9). The online phase (Section III, Eqs. 11–20) has two messages. The device sends a pseudonym $PID = H_2(\lambda_V \| t)$ together with

an authenticated encryption of *PID*, a nonce N_i and the epoch t. The server answers with an authenticated encryption of its nonce N_j, and both sides derive the session key $K = \mathrm{KDF}(k^\star \| N_i \| N_j \| t)$. The evaluation uses per-operation constants (Table III, e.g. $T_H = 0.32$ ms per hash), a per-side cost of $2T_H + 2T_{SE/D} = 11.84$ ms (Table V), i.e. 23.68 ms per session, 1,024 bits over four messages, 320 bits of server storage per device (Table VI) and a simulated network described in Table II.

Observations. The pseudonym is the only field in clear, and the key needed to open the message is specific to the device, so the server must identify the device before it can verify anything. The paper does not say how it does this, nor whether t is a counter or a clock, and its cost model (Table V, Eq. 22) has no term that grows with the number of devices. Its security argument is an informal sketch, and the paper does not claim forward secrecy. Section II describes a different, narrative version of the online phase (Algorithms 1–2) that reuses a static digest without a per-session nonce.

Inference drawn. The Section III exchange is a clean, symmetric-only design and is the specification that the project implements. The missing resolution step and its missing cost are the central gap that the project studies, and the paper's own constants and network are the conditions under which we evaluate every strategy.

Paper 2: Security and Privacy Aspects of Low-Cost Radio Frequency Identification Systems

Authors and year: S. A. Weis, S. E. Sarma, R. L. Rivest and D. W. Engels, 2003 (proceedings 2004) [2]. *Security in Pervasive Computing (SPC 2003)*, LNCS 2802, Springer, pp. 201–212. Link: **[FLAG F5]**.

Summary. The paper analyses the security and privacy risks of low-cost RFID tags, which cannot run conventional public-key cryptography, and proposes hash-based access-control mechanisms. In the *randomized hash lock*, a tag answers each query with a fresh random value r and $H(k\|r)$, so successive answers cannot be linked.

Observations. Because every answer is different, the reader cannot look the tag up directly: it must try the key of every tag it knows, which costs $O(N)$ hash evaluations per identification. The scheme needs no state on the reader beyond the keys, but its identification cost grows linearly with the number of tags.

Inference drawn. This is the simplest private-identification scheme and the baseline for linear search. The project adapts it as strategy **CR**. It needs a fresh r in clear, so it changes the first message of the base protocol.

Paper 3: Cryptographic Approach to "Privacy-Friendly" Tags

Authors and year: M. Ohkubo, K. Suzuki and S. Kinoshita, 2003 [3]. *RFID Privacy Workshop*, MIT, Cambridge, MA, USA. Link: **[FLAG F5]**.

Summary. The paper (often called OSK) proposes tags whose secret state evolves through a one-way hash chain: after each answer the tag replaces its state with its hash, and the answer is a second hash of the state. An attacker who later reads a tag's current state cannot recompute its earlier answers, which gives *forward privacy*.

Observations. To identify a tag the back-end must walk the hash chain of every tag up to a maximum depth m, which costs $O(Nm)$ hash evaluations. If the tag is queried many times without the back-end seeing the answers, the chain moves beyond the search depth.

Inference drawn. Counter epochs in the base protocol behave exactly like an OSK chain, with the chain step replaced by $H_2(\lambda \| t)$ for successive t. Our strategy **O1** (counter epochs with a linear scan over a window) is therefore the OSK search, and the loss of synchronisation that OSK suffers when answers are not seen appears in our results as a permanent lockout after F dropped messages.

Paper 4: Privacy and Security in Library RFID: Issues, Practices, and Architectures

Authors and year: D. Molnar and D. Wagner, 2004 [4]. *Proc. 11th ACM Conference on Computer and Communications Security (CCS)*, pp. 210–219. Link: **[FLAG F5]**.

Summary. The paper studies RFID deployments in libraries and the privacy threats they create, and proposes a private-authentication scheme in which tag secrets are arranged in a tree. A tag answers with one keyed hash per level of the tree, and the reader descends level by level.

Observations. With branching factor δ the reader's work drops to $O(\delta \log_\delta N)$, which scales well. The price is that tags share their internal (non-leaf) keys with other tags, so compromising one tag reveals keys that weaken the privacy of others.

Inference drawn. The project adapts the tree as strategy **MW** with $\delta = 32$. It is the only strategy that is both small and fast in our results, but it changes the first message and weakens the privacy model, which the base protocol does not allow for.

Paper 5: A Scalable and Provably Secure Hash-Based RFID Protocol

Authors and year: G. Avoine and P. Oechslin, 2005 [5]. *Proc. 3rd IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom*

Workshops), pp. 110–114. Link: **[FLAG F5]**.

Summary. The paper keeps the forward privacy of OSK hash chains but makes identification faster with a Hellman-style time–memory trade-off [8]: the back-end precomputes chains over the space of all (tag, state) pairs and stores only their end points, using rainbow tables [9].

Observations. Identification time and memory both become about $O((Nm)^{2/3})$ instead of $O(Nm)$ time. The precomputation must cover every state that a tag may reach, i.e. its whole lifetime.

Inference drawn. The project adapts the trade-off as strategy **TM** (lifetime $m = 128$ epochs, six rainbow tables). Because the table must cover the device's whole lifetime rather than a short window, TM is dominated by the table strategies in our setting.

Paper 6: Reducing Time Complexity in RFID Systems

Authors and year: G. Avoine, E. Dysli and P. Oechslin, 2005 (proceedings 2006) [6]. *Selected Areas in Cryptography (SAC 2005)*, LNCS 3897, Springer, pp. 291–306. Link: **[FLAG F5]**.

Summary. The paper compares private-identification approaches by the time and memory a reader needs to identify one tag. It analyses the tree-based scheme of Molnar and Wagner and quantifies how much privacy is lost when some tags are compromised, and applies the time–memory trade-off to OSK.

Observations. Tree-based schemes are fast but leak privacy under tag compromise. Hash-chain schemes keep forward privacy, and with a time–memory trade-off their cost becomes practical. The comparison is made per tag, not under network load or attack.

Inference drawn. The project's ranked comparison (Table 4.3) follows the same idea, comparing time against memory. It extends the comparison to network scale, to the cost constants of a specific IoT protocol, and to the load created by forged messages.

Paper 7: YA-TRAP: Yet Another Trivial RFID Authentication Protocol

Authors and year: G. Tsudik, 2006 [7]. *Proc. 4th IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*, pp. 640–643. Link: **[FLAG F5]**.

Summary. YA-TRAP makes tag answers depend on a time value instead of a per-tag counter. Because every tag's answer in a given period is predictable to the back-end, the back-end can precompute a table of all expected answers for the current time period

and identify a tag with a single lookup.

Observations. Identification becomes a constant-time lookup after a per-period pre-computation of N hashes. Answers within one time period are fixed, so two sessions of the same tag in the same period produce the same value.

Inference drawn. The project adapts the idea as strategy **O4** (clock epochs with a table extended once per epoch). Our ProVerif analysis confirms the cost of this choice: with clock epochs, two sessions in one epoch share a pseudonym and can be linked.

Table 2.1: Comparative Summary of Literature Survey

Sr. No.	Author(s) & Year	Paper Title	Methodology / Technology Used	Key Observations	Inference Drawn
1	Al-Rasheed et al., 2026 [1]	Low-cost hybrid ECC authentication protocol for trustworthy IoT	Offline TA key derivation; two-message symmetric exchange with epoch pseudonym, AEAD and KDF	Resolution of the pseudonym is required but not specified; cost model constant in N	Specification for the project; source of the gap and of the cost constants
2	Weis et al., 2003 [2]	Security and privacy aspects of low-cost RFID systems	Hash lock; randomized hash lock $(r, H(k\|\|r))$	Unlinkable answers; reader search $O(N)$	Strategy CR (changes M_1)
3	Ohkubo et al., 2003 [3]	Cryptographic approach to "privacy-friendly" tags	Hash-chain tag state (OSK)	Forward privacy; search $O(Nm)$; desynchronises if answers are lost	Counter epochs = OSK search (strategy O1)
4	Molnar and Wagner, 2004 [4]	Privacy and security in library RFID	Tree of shared keys, one keyed hash per level	$O(\delta \log_\delta N)$; compromise of one tag weakens others	Strategy MW (changes M_1)
5	Avoine and Oechslin, 2005 [5]	A scalable and provably secure hash-based RFID protocol	Rainbow time–memory trade-off over OSK chains	$O((Nm)^{2/3})$ time and memory; table covers the whole lifetime	Strategy TM
6	Avoine, Dysli and Oechslin, 2005 [6]	Reducing time complexity in RFID systems	Comparative time/memory analysis; privacy under compromise	Tree schemes leak under compromise; comparison is per tag, no network load	Method for our ranked time–memory comparison
7	Tsudik, 2006 [7]	YA-TRAP: yet another trivial RFID authentication protocol	Time-based answers; per-period precomputed table	$O(1)$ lookup after N hashes per period; same answer within a period	Strategy O4; linkability within an epoch

2.3 Research gaps

The review shows that the base protocol relies on a step that it neither specifies nor costs, and that the RFID literature, which offers solutions to that step, has never evaluated them under the conditions of this protocol. Table 2.2 lists the gaps and how the project responds to each.

Table 2.2: Research Gap Analysis

Sr. No.	**Existing Approach / System**	**Identified Limitation / Research Gap**	**Proposed Project's Approach**
1	Al-Rasheed et al. [1], Section III	The server must resolve *PID* before any check, but the resolution step and the epoch semantics are not specified	Prove from the specification that resolution is required; map the open choices (epoch semantics, resolution method)
2	Al-Rasheed et al. [1], Tables III–V, Eq. 22	Cost model charges a constant $2T_H + 2T_{SE/D}$ per session, with no term that grows with N	Measure probe counts for seven strategies; price them at $T_H = 0.32$ ms in an OMNeT++ model of the paper's Table II network
3	RFID private identification [2, 3, 4, 5, 7]	Evaluated for RFID tags, not under the cost constants and network of an IoT authentication protocol	Adapt each scheme to the protocol (O1–O4, TM, CR, MW) and rank them by latency and storage at up to 1,000 devices
4	Comparative study [6]	Time and memory per tag only; no network load and no forged traffic	Add 5% adversaries sending forged M_1 at 1 Hz and model server queueing
5	Base protocol, threat model	Desynchronisation of counter epochs under dropped messages is not considered	Demonstrate the lockout after $F = 8$ dropped messages and test it for each strategy
6	Base protocol, security sketch	Informal; does not state which properties depend on epoch and replay choices	ProVerif models of secrecy, agreement, injectivity, forward secrecy and unlinkability for every correct resolver

The most significant gap is the first two together: a required step with no specifica-

tion and no cost. A server cannot authenticate a device that it has not yet identified, so the protocol's headline figure of 23.68 ms per session is only valid if resolution is free, which, as the RFID literature shows, it is not unless the server spends memory on pre-computation. This gap justifies the project, because it can be closed with measurement rather than speculation: the protocol, its constants and its network are all published, and the candidate solutions are well understood.

2.4 Conclusion

The literature shows a clear pattern. Lightweight IoT protocols such as [1] hide the device behind an unlinkable pseudonym and evaluate their cost per session as if the server already knew the device. RFID research shows that identifying a device from an unlinkable answer costs $O(N)$ work unless the server precomputes, and offers a family of trade-offs between time, memory, message format and privacy: hash locks, hash chains, key trees, time–memory trade-offs and time-based tables. The common limitations are linear search cost, desynchronisation of counter-based state, and loss of privacy when keys are shared or answers repeat.

The research gaps of Section 2.3 are therefore that the base protocol neither specifies nor costs resolution, and that the known solutions have not been compared under its conditions or analysed for their effect on its security properties. These findings set the objectives of the project and justify its methods: an implementation that is checked against independent vectors, measured probe counts checked against closed forms, a network simulation under the paper's own constants, and symbolic verification. The next chapter presents the methodology built on these choices.

CHAPTER: 3

PROPOSED METHODOLOGY

Chapter 3

Proposed Methodology

3.1 Introduction

The proposed system is an open, measured study of pseudonym resolution in the protocol of [1]. It consists of a reference implementation of the protocol whose server can be configured with any of seven resolvers, instrumentation that counts every hash evaluation the server makes, a network model that prices those counts with the paper's own constants, and symbolic models that check the protocol's security properties for every correct resolver.

The methodology follows the order in which each result depends on the previous one:

1. **Specification and conformance.** Implement Section III of [1] and fix its message layout with independent reference vectors, in Python and in C.
2. **Gap analysis.** Using only the specification and the vectors, show that resolution is required and identify the choices it leaves open.
3. **Design space.** Implement seven resolution strategies along those choices, each with a closed-form cost.
4. **Measurement and validation.** Count probes for honest and forged pseudonyms for $N = 1$ to $10{,}000$ and check the counts against the closed forms.
5. **Network evaluation.** Feed the measured cost model into an OMNeT++ model of the paper's Table II network and rank the strategies.
6. **Failure modes and formal verification.** Test desynchronisation and replay, and verify secrecy, agreement and unlinkability in ProVerif.

Each phase serves one of the objectives of Section 1.3, and every number reported later can be regenerated from the repository.

3.2 Requirement Gathering

The project is a research study of a published protocol, so its requirements were gathered by *document analysis* rather than by surveys or interviews. The team studied the base paper [1] equation by equation (Eqs. 8–24, Tables II–VI) and recorded, for every equa-

tion, the code that implements it and the evidence that checks it (the equation-to-code matrix in `docs/conformance.md`). Every decision that the paper does not fix (epoch semantics, search window, failure threshold, nonce length, persistence of the epoch) was written into an *assumption register* together with its basis. The RFID literature reviewed in Chapter 2 supplied the candidate resolution strategies and their known costs. The initial requirements were written up as a Product Requirements Document (PRD, Draft v0.1, 25 August 2026), which fixed the decision to implement Section III of the paper and defined functional, non-functional and security requirements with identifiers (FR, NFR, SEC) that still label the tests. They were then refined in the project review cycle: the project notes record the mentor's conditions that the base paper be implemented in OMNeT++ and that its published results be reproduced before any new measurement is compared with them, which led to the network model used in this report.

3.2.1 Functional Requirements

FR-1 **Enrolment.** A trusted authority shall derive each entity's long-term secret $\lambda_X = H_1(ID_X \| s)$ (Eq. 8), issue credentials to devices and servers, and share peer secrets so that each pair can derive $k^\star$ (Eq. 9).

FR-2 **Request.** A device shall build $M_1 = \langle PID, \mathrm{AEAD}_{k^\star}(PID \| N_i \| t) \rangle$ with $PID = H_2(\lambda_V \| t)$ (Eqs. 11–15).

FR-3 **Resolution.** A server shall map an incoming *PID* to an enrolled device and an epoch using a configurable resolver, and shall count the hash evaluations it makes.

FR-4 **Verification and response.** The server shall decrypt and verify M_1, check that the decrypted *PID* and t match, and return $M_2 = \langle ID_R, \mathrm{AEAD}_{k^\star}(N_j \| t;\ PID \| N_i) \rangle$ (Eqs. 17–19).

FR-5 **Key agreement.** Both sides shall derive the same session key $K = \mathrm{KDF}(k^\star \| N_i \| N_j \| t)$ (Eq. 16).

FR-6 **Replay and abuse handling.** The server shall reject a replayed M_1, and both sides shall block a peer after three failed attempts.

FR-7 **Strategies.** The server shall support strategies O1–O4 and TM; strategies CR and MW shall be modelled at the resolution level.

FR-8 **Measurement.** Scripts shall measure probes per honest and forged message and the resolution state for $N = 1$ to 10,000, and write a cost model for the simulator.

FR-9 **Simulation and verification.** An OMNeT++ model shall price the cost model under the paper's network parameters, and ProVerif models shall check the security properties.

FR-10 **Demonstration.** Command-line tools shall enrol entities, run a server, authenticate a device and demonstrate replay rejection; the C device port shall interoperate

with the Python server.

3.2.2 Non Functional Requirements

- **Security.** Standard primitives only: AES-256-GCM [13], HKDF-SHA256 [11], SHA-256 [12]; nonces from a cryptographically secure generator; constant-time comparison of each candidate pseudonym; secrets never printed (only a fingerprint of the session key).
- **Conformance.** The Python and C implementations shall reproduce reference vectors generated by code that shares nothing with them, byte for byte.
- **Reliability and quality.** At least 90% statement coverage of the protocol and crypto packages, enforced in continuous integration on Python 3.11 and 3.12.
- **Performance.** Resolution cost shall be reported in probes and priced at the paper's $T_H = 0.32$ ms, so that results are comparable with the paper's 23.68 ms per session.
- **Reproducibility.** Pinned tool versions (OMNeT++ 6.4.0, ProVerif 2.05), committed data and logs, and one command per result.
- **Portability.** The device side shall run on an 80 MHz ESP8266 without hardware cryptography.
- **Scalability.** Measurements shall cover up to 10,000 devices per server and simulations up to 1,000 devices per network.

3.2.3 User Requirements

- **Protocol designers and researchers** need to know which properties of the protocol depend on the resolution choice, and what term the cost model is missing.
- **Implementers and deployers** need a recommendation: which strategy to choose for their network size, memory budget and threat model, and what each choice costs.
- **Reviewers and examiners** need to reproduce every number from the released code and data, and to inspect results without installing the simulator or verifier.
- **Device owners** need devices that do only symmetric work online and that are not locked out by an attacker who drops a few messages.

3.3 Proposed Algorithms

The base protocol

In the offline phase the trusted authority computes, for each entity X and a master secret s,

$$\lambda_X = H_1(ID_X \| s), \qquad k^\star = \mathrm{KDF}(\lambda_V \| \lambda_R), \tag{3.1}$$

so that device V and server R share the pairwise key $k^\star$. Online, in epoch t, the device samples a nonce N_i and sends

$$M_1 = \langle PID,\ \mathrm{AEAD}_{k^\star}(PID \| N_i \| t) \rangle, \qquad PID = H_2(\lambda_V \| t). \tag{3.2}$$

The server resolves PID to (V, t), decrypts and verifies M_1, samples N_j, and returns

$$M_2 = \langle ID_R,\ \mathrm{AEAD}_{k^\star}(N_j \| t;\ PID \| N_i) \rangle, \tag{3.3}$$

where $PID \| N_i$ is associated data. Both sides derive

$$K = \mathrm{KDF}(k^\star \| N_i \| N_j \| t). \tag{3.4}$$

We instantiate AEAD as AES-256-GCM, KDF as HKDF-SHA256 and H as SHA-256. Table 3.1 summarises the notation.

Table 3.1: Notation

Symbol	Meaning
V, R	Device and server
λ_X	Long-term secret of entity X (Eq. 8)
$k^\star$	Pairwise key of a device–server pair (Eq. 9)
t, PID	Epoch, and pseudonym $H_2(\lambda_V \| t)$ (Eq. 11)
N_i, N_j	Device and server nonces (128 bits)
N, n	Devices in the network; devices enrolled at one server
L, F, W	Backward and forward epoch tolerance ($L = 16$, $F = 8$) and window $W = L + F + 1 = 25$
W_t	Clock-epoch drift window ($W_t = 3$)
m	Device lifetime in epochs for TM ($m = 128$); $S = Nm$
δ, d	Branching factor and depth of the key tree ($\delta = 32$)
T_H	Cost of one hash, 0.32 ms (Table III of [1]); one *probe* is one H_2 evaluation

Why resolution is required

Each step of the argument uses only the specification or a fact confirmed by the reference vectors:

1. M_1 carries *PID* in clear; t and N_i are only inside the ciphertext under $k^\star$ (Eqs. 13–15, confirmed byte for byte).
2. $k^\star = \mathrm{KDF}(\lambda_V \| \lambda_R)$ is specific to the device (Eqs. 9, 12).
3. So the server cannot decrypt M_1 until it knows which device sent it.
4. The paper requires that *PID* cannot be linked to V without λ_V (its Section III-J), so the identity cannot be read from M_1 either.
5. So every correct server must map *PID* to (V, t) by computing pseudonyms from the secrets it holds.

Two choices are left open and together decide the cost: the *epoch semantics* (a per-device counter or a clock) and the *resolution method* (an on-demand search or a table prepared in advance). Figure 3.1 shows where the step sits.

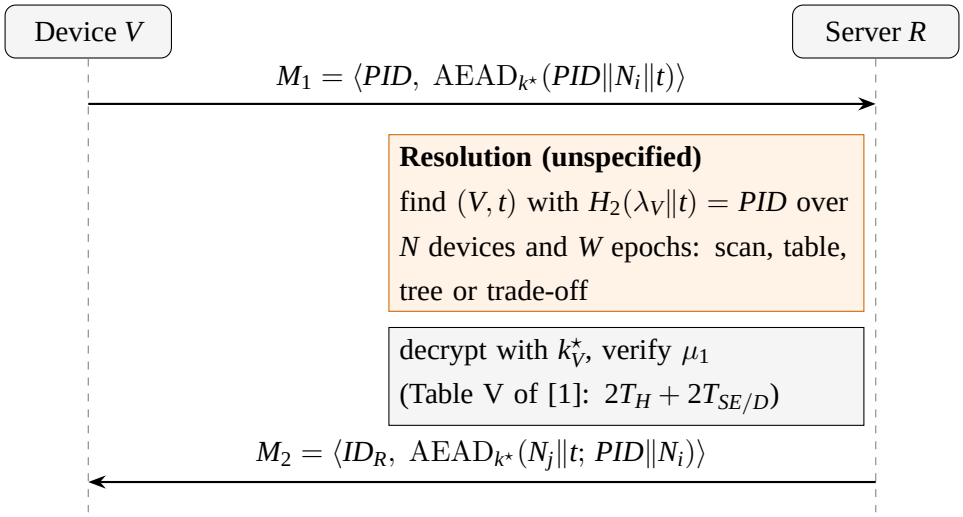


Figure 3.1: Where pseudonym resolution sits in the exchange. The server needs $k_V^\star$ to open M_1, but V is hidden behind *PID*.

The seven resolution strategies

The cost of resolution is counted in *probes*, one probe being one evaluation of H_2 (or the equivalent keyed hash). The first five strategies keep the wire format of M_1; CR and MW need a fresh value r in clear and therefore change M_1.

O1 – counter epochs, linear scan (the behaviour of our original implementation; the OSK search [3]). For each enrolled device the server probes the epochs from $\ell - L$ to $\ell + F$ around the last accepted epoch ℓ.

```
function ResolveO1(PID):
  for each enrolled device V with secret λ_V and last accepted epoch ℓ_V do
    for t = ℓ_V − L to ℓ_V + F do
      if H_2(λ_V‖t) = PID then            // constant-time compare
        if t ≤ ℓ_V then raise ReplayDetected
        return (V, t)
  raise UnknownPeer                          // a forged PID costs N·W
    probes
```

O2 – counter epochs, precomputed table. The server keeps a dictionary from each device's next F pseudonyms to (V, t). Resolution is a lookup; after an epoch is accepted the table slides, which costs one probe in the normal case.

```
setup: for each V: for t = ℓ_V+1 to ℓ_V+F: T[H_2(λ_V‖t)] = (V, t)
function ResolveO2(PID):
  if PID in T then return T[PID] else raise UnknownPeer   // no hash on a
    miss
function AcceptO2(V, t):
  remove entries of V with epoch ≤ t;  add H_2(λ_V‖t′) for new t′ up to t+F
```

O3 – clock epochs, linear scan. The epoch is $t = \lfloor \text{clock}/\text{epoch length} \rfloor$; the server probes every device for each epoch in a drift window of $W_t = 3$ epochs, and keeps a cache of accepted nonces to detect replays.

O4 – clock epochs, per-epoch table (the time-based idea of YA-TRAP [7]). At each epoch boundary the server adds every device's pseudonym for the new epoch (N probes per epoch) and drops expired ones; resolution is a lookup with no probes.

TM – rainbow time–memory trade-off [5, 6, 8, 9]. Over the space of $S = Nm$ points (device, epoch) with lifetime $m = 128$, the server builds six rainbow tables of $2S^{2/3}$ chains of length $\lceil S^{1/3} \rfloor$ and stores only chain end points. A lookup regenerates chains from the received *PID*.

CR – randomized hash lock [2]. The device sends $(r, H(k\|r))$ for a fresh 128-bit r; the server tries every key.

MW – key tree [4]. Keys form a tree with branching factor $\delta = 32$; the device sends one keyed hash per level and the server descends level by level, trying at most δ keys per level.

Table 3.2 gives the closed-form cost of each strategy, which the measurements in Chapter 4 are checked against.

Table 3.2: Design space of resolution strategies. One table entry is charged at 40 B; "probes" are expected H_2 evaluations per message in steady state.

	Epoch / M_1	**Probes, honest**	**Probes, forged**	**Pre-compute**	**Resolution state**	**Dropped M_1**	**Link-ability**
O1	counter / unchanged	$\frac{(N-1)W}{2} + L + 2$	NW	none	$8N$ B	lockout after F	none
O2	counter / unchanged	1 (slide)	0	NF once	$(40F + 8)N$ B	lockout after F	none
O3	clock / unchanged	$\frac{(N-1)W_t}{2} + 1$	NW_t	none	nonce cache	none	same epoch
O4	clock / unchanged	0	0	N per epoch	$40W_tN$ B + nonce cache	none	same epoch
TM	counter / unchanged	≈ 0.5–$0.7\,S^{2/3}$ (measured)	$\approx 4\,S^{2/3}$ (measured)	$\approx 12S$ once	$\approx 50\,S^{2/3}$ B	none until $t = m$	none
CR	none / $+r$ (128 b)	$\frac{N+1}{2}$	N	none	none	none	none
MW	none / $+r$, $+(d-1)$ tags	$\leq \delta d$	$\leq \delta$	none	32× internal nodes	none	after compro-mise

Evaluation method

The network evaluation follows the paper's conditions and changes only the resolution term:

- **Simulator and environment:** OMNeT++ 6.4 [10] (Cmdenv) with Table II of [1]: a 1500 × 1500 m area, 10 Mbps, 500 m range, 75.6 mW, a 15 μs quantum, and servers at 5% of the devices.
- **Protocol cost:** Tables III–V of [1]: $2T_H + 2T_{SE/D} = 11.84$ ms per side, 1,024 bits over four messages, and $T_H = 0.32$ ms per probe.
- **Resolution cost:** for each server, the probes for its number of enrolled devices n are read from the measured cost model (exact for O1–O4 and CR, interpolated between measured points for TM and MW). CR and MW also add their extra M_1 bits.
- **Assumptions** (not fixed by the paper): 5% adversaries, each sending forged M_1 at 1 Hz; a 10 s authentication window; each server enrols every device in its range; first-come-first-served processing; one O4 table extension per server per window.
- **Runs:** five seeds per point, $N \in \{1, 10, 100, 200, \ldots, 1000\}$, with and without adversaries: 960 runs in total, reported as means.

For formal verification, the exchange is modelled in ProVerif 2.05 [14] in the symbolic model with an active attacker, with unbounded servers, honest and corrupt devices and sessions. The server may resolve a pseudonym to *any* enrolled device and must then de-

crypt and check $PID = H_2(\lambda_V \| t)$, so every property that holds, holds for every resolver that returns a correct device. Unlinkability is checked as observational equivalence [15] in the style of [16].

3.4 Gantt chart for Timeline

Figure 3.2 shows the project schedule. The bars are taken from the dates of the corresponding commits in the project repository (first and last commit of each activity). **[FLAG F10]**

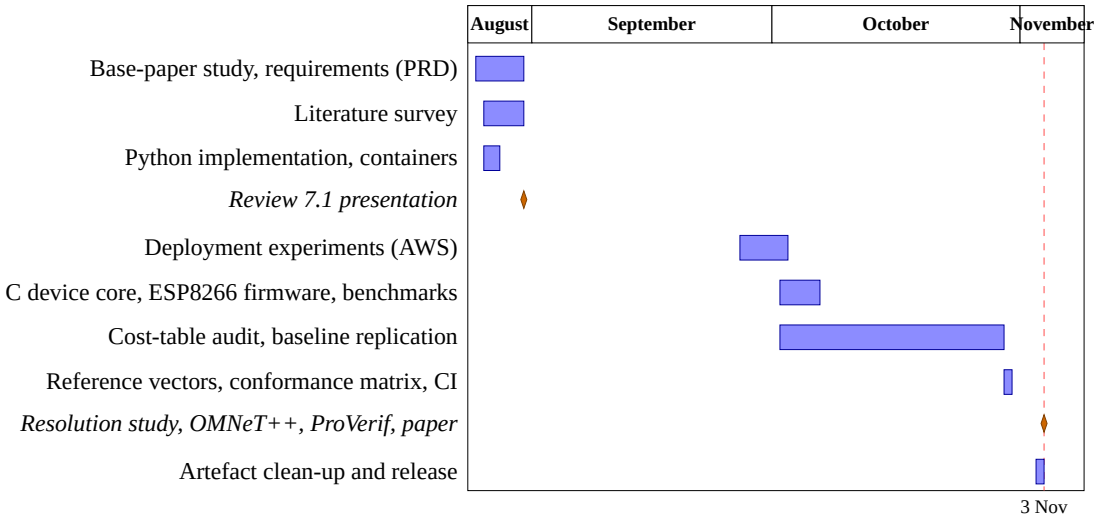


Figure 3.2: Gantt chart of the project timeline (from repository commit dates)

3.5 Architectural Diagram

Figure 3.3 shows the layered architecture of the system. The *crypto* layer wraps the primitives and contains no protocol logic. The *protocol* layer contains transport-agnostic state machines for the trusted authority, device and server, the message formats and the pluggable resolvers. The *storage* layer holds the session log (replay window and blocklist) and a demonstration key store. The *demo* layer adds length-prefixed TCP framing and the command-line tools. The C device core reuses the same wire format. The *evaluation* layer measures, simulates and verifies, and feeds the figures and tables of the paper and of this report.

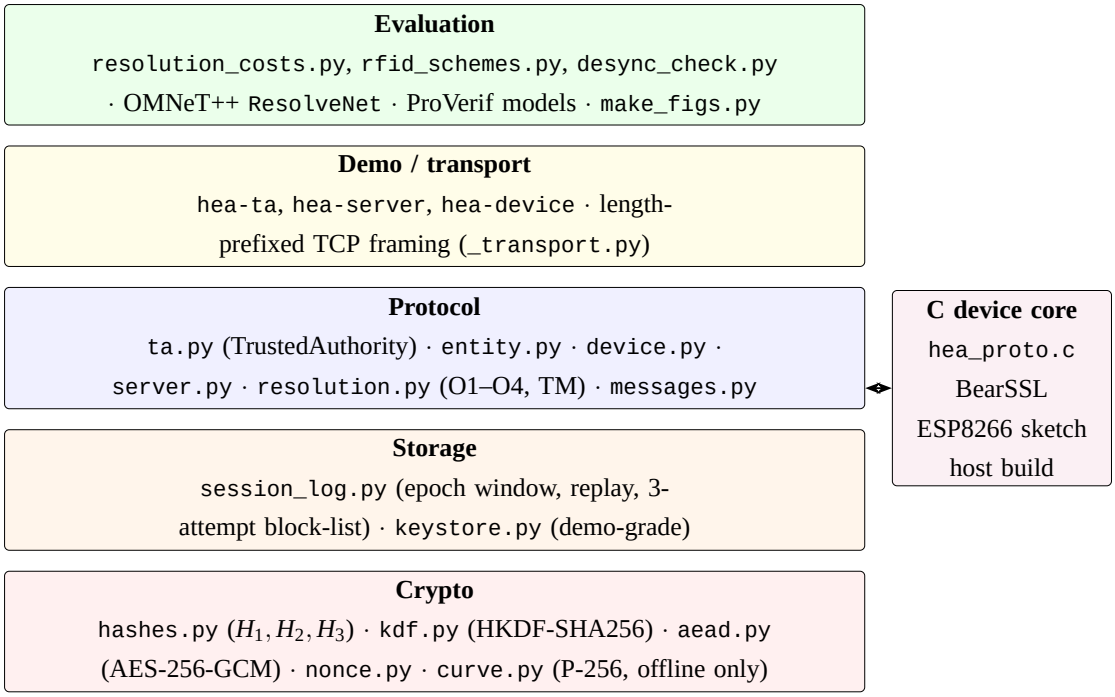


Figure 3.3: Layered architecture of the implementation and evaluation

3.5.1 UML Diagrams

Use-case diagram. Figure 3.4 shows the actors and the services of the system. The trusted authority enrols entities offline. A device requests authentication and derives the session key; the server resolves the pseudonym, verifies the request and responds. An adversary may replay, forge or drop messages. The researcher runs the measurements, simulation and verification.

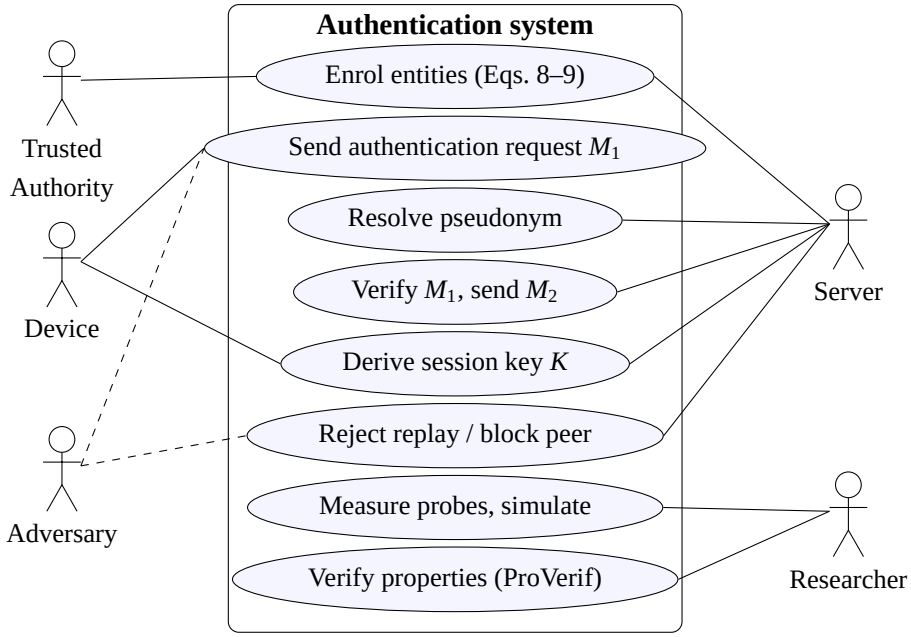


Figure 3.4: Use-case diagram

Class diagram. Figure 3.5 shows the main classes of the Python implementation. `Device` and `Server` share the `Entity` base class, which caches pairwise keys and implements the coverage and access-control gates (Eqs. 10, 23). A `Server` owns a `SessionLog` and one `Resolver`; the resolver subclasses implement the strategies O1–O4 and TM.

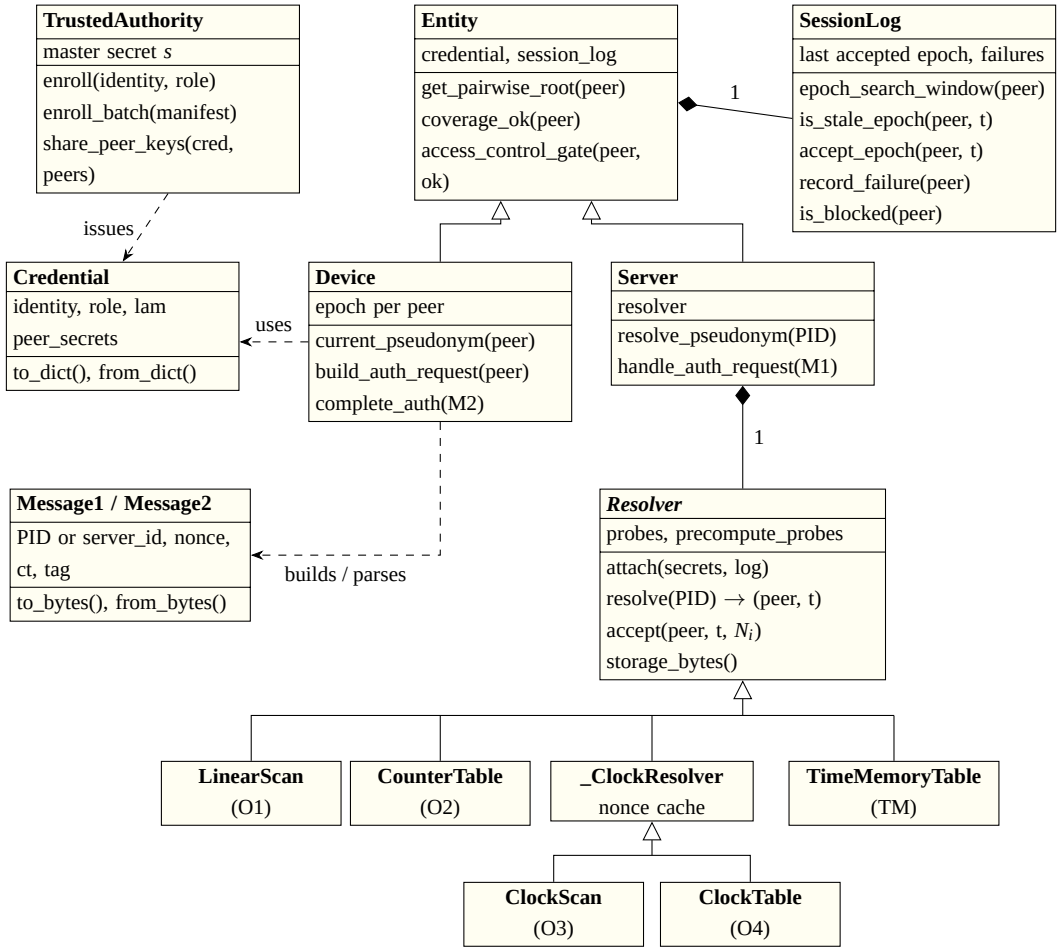


Figure 3.5: Class diagram of the Python implementation

Sequence diagram. Figure 3.6 shows one successful authentication, including the resolver and the session log inside the server.

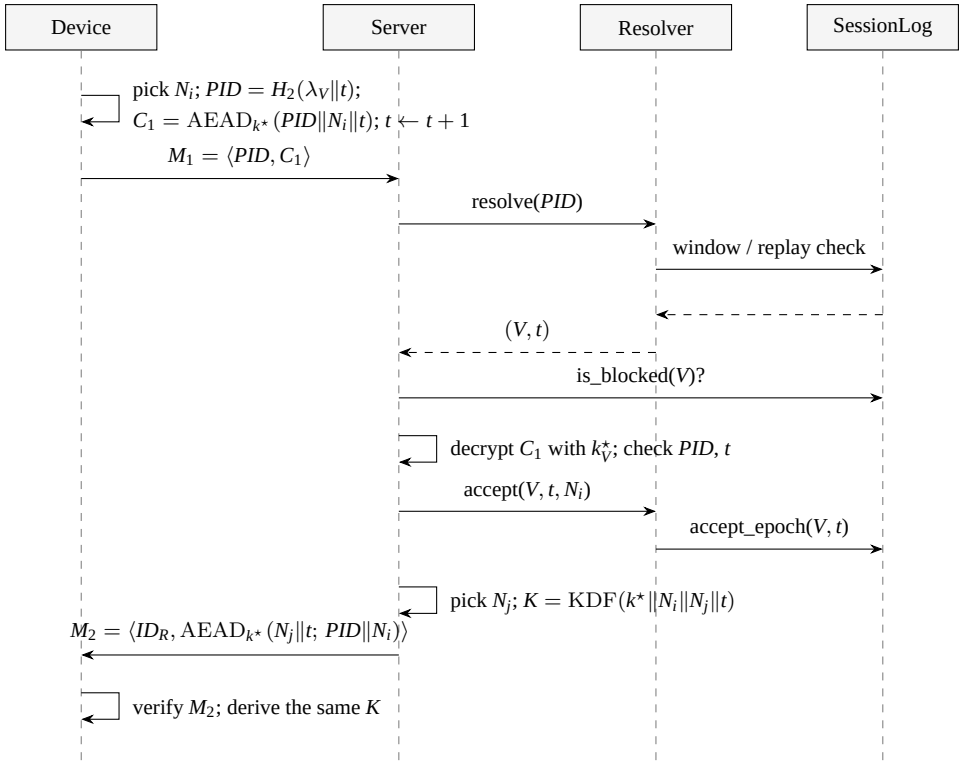


Figure 3.6: Sequence diagram of one authentication

3.5.2 Block Diagram

Figure 3.7 shows the blocks of the system and the flow between them: the offline enrolment, the online exchange between a device (Python or ESP8266) and the server with its pluggable resolver, and the evaluation pipeline that turns measured probe counts into the simulation results, figures and tables.

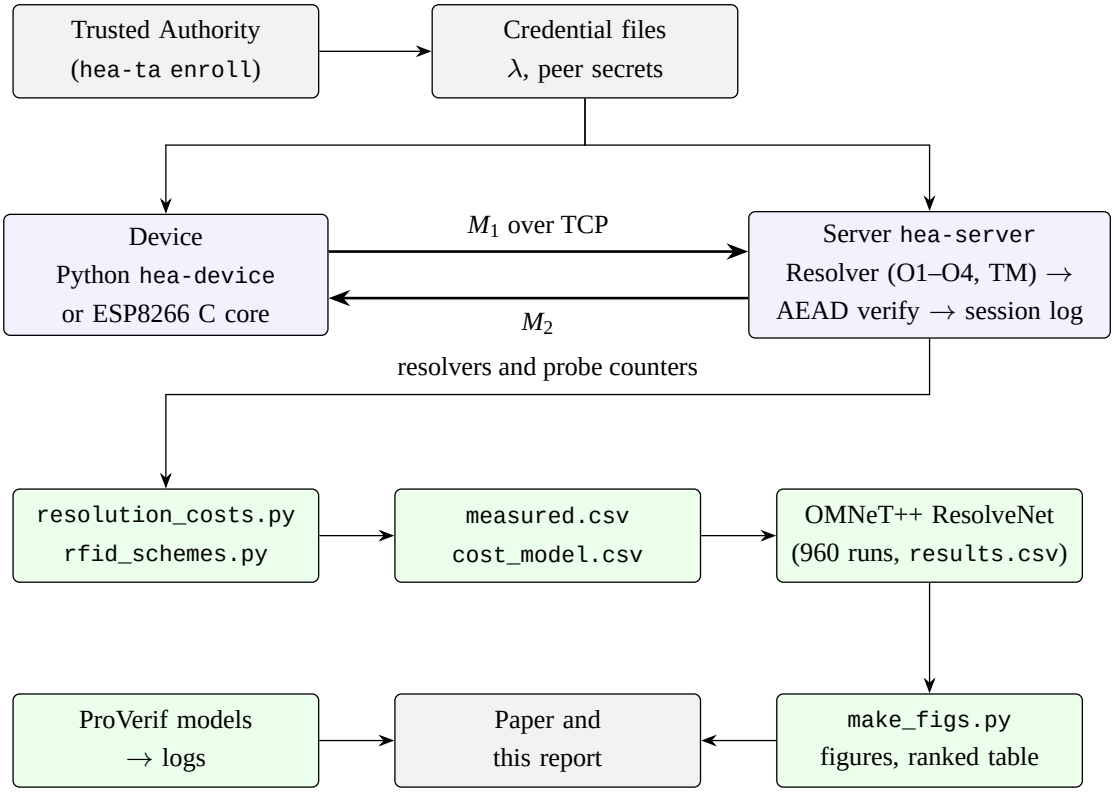


Figure 3.7: Block diagram of the system

3.5.3 Data Flow Diagram

Figure 3.8 is the context-level (level 0) DFD: the system exchanges enrolment data with the trusted authority, requests and responses with devices, and receives forged or replayed messages from an adversary. Figure 3.9 refines it into level 1 processes and data stores.

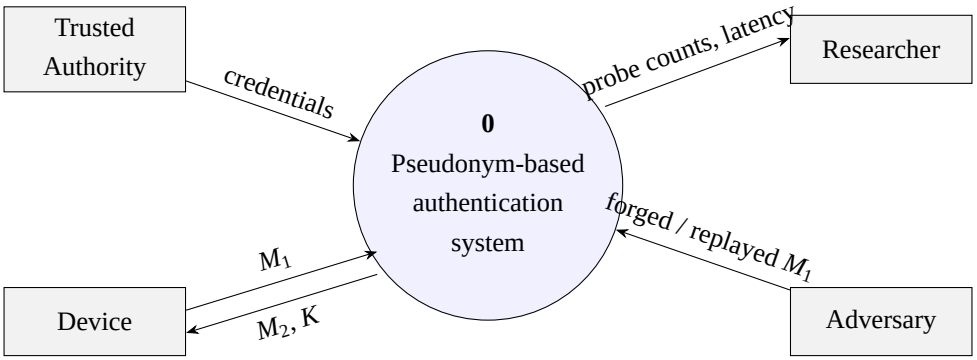


Figure 3.8: Data flow diagram, level 0 (context)

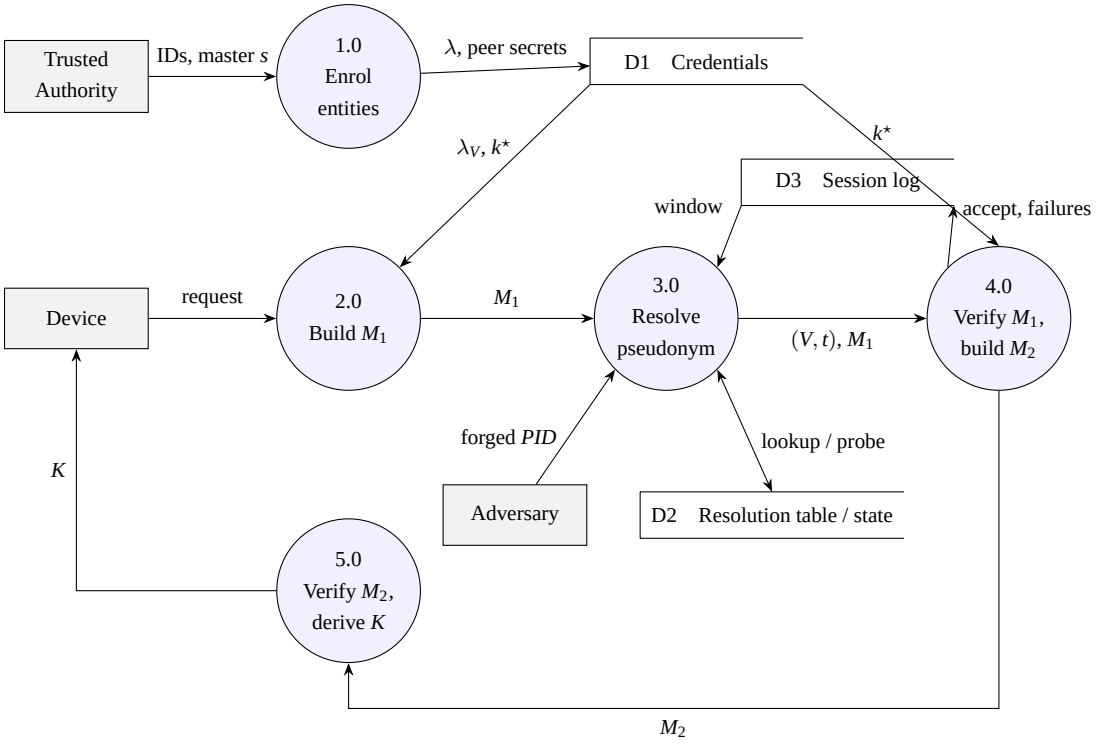


Figure 3.9: Data flow diagram, level 1

3.5.4 Timeline Chart

The Gantt chart above shows activities and their durations; Figure 3.10 instead shows the project's milestones, i.e. the dates on which each deliverable became available in the repository.

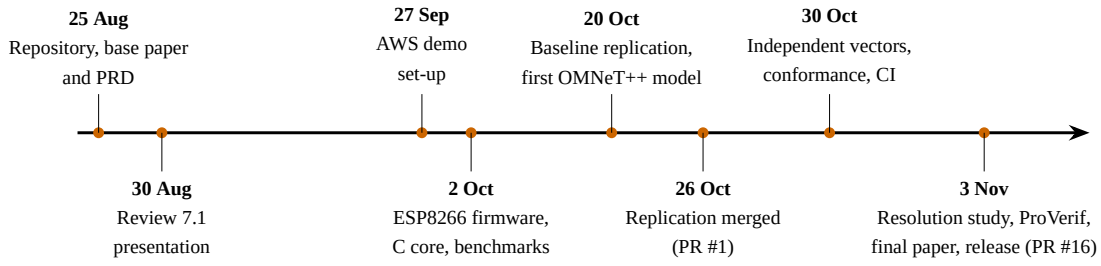


Figure 3.10: Timeline chart of project milestones (2026)

3.6 Hardware Requirements

Table 3.3: Hardware requirements

Component	**Specification / use**	**Status**
Development computers	Laptops of the team members, used for development, measurement and as the server in the demonstration. The OMNeT++ 6.4.0 toolchain was built natively on a Mac with Apple clang, and ProVerif 2.05 with Homebrew OCaml (project notes); the model and specification of each laptop are not recorded. **[FLAG F11]**	Already available
Machine used for this report's re-runs	12th Gen Intel Core i5-12500H, 16 logical CPUs, 15 GiB RAM, Linux 6.18 (tests, C vector check, interoperability, desynchronisation, probe sweep and figure generation)	Already available
ESP8266 development boards (NodeMCU), 2	Device side of the protocol in C; 80 MHz stock clock (160 MHz selectable), Wi-Fi, no hardware cryptography; epoch persisted to flash. Two boards were used (chip IDs 00d37d28 and 00d377aa); the port can also be verified without hardware through the host build. **[FLAG F7]**	Additionally required
Network	Wi-Fi access point / LAN connecting the board and the server laptop	Already available

3.7 Software Requirements

Table 3.4: Software requirements

Software	**Version and purpose**
Operating system	Linux or macOS (CI runs on Ubuntu)
Python	3.11 or newer (CI: 3.11 and 3.12; this report's re-runs: 3.14.9); reference implementation, measurement and figure scripts
`cryptography`	$\geq$ 41.0 (50.0.2 in the re-runs): AES-256-GCM, HKDF-SHA256, SHA-256, P-256
pytest, pytest-cov, Hypothesis	Unit, property and adversarial tests; coverage gate of 90%
matplotlib	Figures (paper and this report)
C compiler (GCC/Clang), Make	Host build of the C device core and the vector check
BearSSL (ESP8266 Arduino core fork)	AES-256-GCM, SHA-256 and HKDF in the C device core
PlatformIO, Arduino framework	Building and flashing the ESP8266 firmware
OMNeT++	6.4.0: network simulation (ResolveNet, 960 runs)
ProVerif	2.05 (built from source with OCaml 5.5.2): symbolic verification
Git, GitHub, GitHub Actions	Version control, code review through pull requests, continuous integration
LaTeX (Tectonic 0.17.0, XeLaTeX)	Paper and this report

3.8 Cost Estimation

Table 3.5: Cost estimation

Sr. No.	Item / Resource	Description	Estimated Cost (₹)
1	Hardware	Team laptops (existing resource); two ESP8266 NodeMCU boards for the device port	Laptops: existing; boards: **[FLAG F7]**
2	Software / Tools	Python and libraries, GCC, BearSSL, PlatformIO, ProVerif, OMNeT++ (free for academic use), LaTeX	₹ 0 (free/open-source)
3	Cloud / Hosting	GitHub repository and GitHub Actions (free tier). An earlier field test (27 Sep 2026) ran on two on-demand AWS t4g.nano instances (US$0.0042 per hour each, plus two 8 GB gp3 volumes), terminated after use; it is not part of the final evaluation.	₹ 0 for the final work; AWS: **[FLAG F8]**
4	APIs / Services	None used	₹ 0
5	Other Expenses	Publication or conference fees, printing and binding of this report	**[FLAG F9]**
	Total Estimated Cost		**[FLAG F9]**

The estimate follows the department's instructions: resources that were already available (the team's laptops and the network) are listed as existing resources with no purchase cost, and all software is free or open source and is listed at ₹ 0. The only project-specific purchase is the pair of ESP8266 development boards used to run the device side of the protocol on real hardware; the study itself (measurement, simulation and verification) needs no paid resource. No recurring expense is incurred for the final work.

3.9 Conclusion

This chapter set out a methodology in which every phase builds on a checked result: the protocol is implemented from Section III of [1] and pinned down with independent vectors; the need for resolution is derived from the specification alone; seven strategies drawn from RFID private identification are implemented with closed-form costs; their probe counts are measured, validated and priced under the paper's own constants in an OMNeT++ model of its network; and the security properties are checked in ProVerif.

Requirements were gathered by analysing the specification and recorded in a conformance matrix and an assumption register. The layered architecture separates primitives, protocol logic, storage, transport and evaluation, so that resolvers can be exchanged without touching the protocol. All software is free, the hardware already existed apart from an inexpensive development board, and the schedule was met, which provides a sound foundation for the implementation and results in the next chapter.

CHAPTER: 4

IMPLEMENTATION & RESULTS

Chapter 4

Implementation & Results

4.1 Introduction

This chapter describes how the methodology of Chapter 3 was turned into working software and what it produced. The implementation has four parts: the Python reference implementation of the protocol with its pluggable resolvers; the C port of the device side for the ESP8266; the measurement and simulation pipeline; and the ProVerif models. The results are evaluated in the same order as the objectives: conformance of the implementations to the independent vectors, correctness of the test suite, probe counts against their closed forms, session latency and storage in the network model, desynchronisation behaviour, and the formal security properties.

Unless stated otherwise, the numbers come from the data committed in the project repository (commit `e21993a`), which is the data used in the project's paper. For this report, the test suite, the C vector check, the interoperability check, the desynchronisation check, the probe-count sweep and the figure generation were re-run on the machine described in Table 3.3; their logs are kept in the `results/` folder next to this report. The OMNeT++ sweep and the ProVerif models were not re-run for this report; their committed outputs are used.

4.2 Implementation

Python reference implementation

The package `hybrid_ecc_auth` follows the layered architecture of Figure 3.3. The *crypto* layer instantiates the paper's primitives: H_1, H_2 and H_3 with SHA-256, the KDF with HKDF-SHA256, AEAD with AES-256-GCM, and 128-bit nonces from the operating system's secure generator. P-256 scalar multiplication is used only in the offline phase. The *protocol* layer implements the trusted authority (`TrustedAuthority.enroll`), the device (`Device.build_auth_request`, `complete_auth`) and the server (`Server.handle_auth_request`), with field names that follow the paper's notation so that each line can be traced to its equation. Messages are versioned and serialised as JSON with base64 fields; an M_1 is 216 bytes on the

wire. Listing 4.1 shows the server's handling of M_1: resolution comes first, then the block-list check, the AEAD decryption, and the check that the decrypted pseudonym and epoch match the header.

```
def handle_auth_request(self, msg1_bytes: bytes) -> AuthResult:
    msg1 = Message1.from_bytes(msg1_bytes)
    peer_identity, epoch = self.resolve_pseudonym(msg1.pid)
    if self.session_log.is_blocked(peer_identity):        # fail fast
        raise BlockedEntity(...)
    k_star = self.get_pairwise_root(peer_identity)
    try:
        plaintext = aead.decrypt(k_star, msg1.gcm_nonce, msg1.ciphertext, msg1.
    tag)
    except InvalidTag as exc:
        self.session_log.record_failure(peer_identity)
        raise IntegrityError(...) from exc
    pid_in_plaintext, n_i = plaintext[:32], plaintext[32:-8]
    epoch_in_plaintext = int.from_bytes(plaintext[-8:], "big")
    if pid_in_plaintext != msg1.pid or epoch_in_plaintext != epoch:
        self.session_log.record_failure(peer_identity)
        raise IntegrityError(...)
    self.resolver.accept(peer_identity, epoch, n_i)
    self.session_log.record_success(peer_identity)
    n_j = generate_nonce(self.nonce_bits)
    session_key = derive_session_key(k_star, n_i, n_j, epoch)
    gcm_nonce2, ct2, tag2 = aead.encrypt(k_star, n_j + epoch.to_bytes(8, "big")
    ,
                                         aad=msg1.pid + n_i)
    ...
```

Listing 4.1: Server handling of M_1 (`protocol/server.py`, abridged)

The *storage* layer's `SessionLog` keeps, per peer, the last accepted epoch and the search window $[\ell - L, \ell + F]$ with $L = 16$ and $F = 8$, rejects stale epochs as replays, and block-lists a peer after three failed attempts. The *demo* layer adds length-prefixed TCP framing and the command-line tools `hea-ta`, `hea-server` and `hea-device`; the device tool can replay its last M_1 to demonstrate replay rejection.

Pluggable resolvers

Every resolver implements the interface of Listing 4.2: `attach` is called once with the enrolled secrets, `resolve` for every M_1, and `accept` after M_1 has been verified. Every H_2 evaluation goes through one method, which counts it either as a probe (work for an incoming message) or as precomputation. Strategies O1–O4 and TM live in `protocol/resolution.py` and run inside the real hand-

shake; CR and MW change M_1 and are implemented at the resolution level in `evaluation/resolution/rfid_schemes.py`.

```
class Resolver:
    def _h2(self, lam, epoch, *, precompute=False):
        if precompute: self.precompute_probes += 1
        else:          self.probes += 1
        return h2_pseudonym(lam, epoch)

class LinearScan(Resolver):                       # O1
    def resolve(self, pid):
        log = self.session_log
        for peer_identity, peer_lam in self.peer_secrets.items():
            for epoch in log.epoch_search_window(peer_identity):
                if hmac.compare_digest(self._h2(peer_lam, epoch), pid):
                    if log.is_stale_epoch(peer_identity, epoch):
                        raise ReplayDetected(...)
                    return peer_identity, epoch
        raise UnknownPeerError("pseudonym did not resolve to any known, in-
    window peer")

class CounterTable(Resolver):                     # O2
    def resolve(self, pid):
        hit = self._table.get(pid)                # a lookup costs no hash
        if hit is None:
            raise UnknownPeerError("pseudonym is not in the precomputed table")
        return hit
```

Listing 4.2: Resolver interface and strategies O1 and O2 (`protocol/resolution.py`, abridged)

C device port for the ESP8266

The device half of the protocol was ported to portable C (`implementation/c/lib/hea_proto`) on top of BearSSL, the library shipped with the ESP8266 Arduino core. It provides `hea_init` (derives $k^{\star}$), `hea_build_m1` and `hea_complete` (verifies M_2 and derives K), and speaks the same framed-JSON wire format as the Python server. The Arduino firmware joins Wi-Fi, runs the handshake in a loop, prints per-step timings and a session-key fingerprint, and persists the epoch to flash. The same core builds natively on Linux and macOS, so it can be checked without the board: `vector_test` runs it against the reference vectors, and `tools/interop.py` runs it against the unmodified Python server.

Independent reference vectors

A separate generator, which shares no code with either implementation, computes the pairwise key, the pseudonym at epochs 0 and 3, the exact wire bytes of M_1 and M_2, and the session key, together with tampered and replayed messages that must be rejected. These vectors fix the message layout on which the gap argument of Section 3.3 relies.

Measurement, simulation and verification

`resolution_costs.py` builds each server in steady state for 15 values of N from 1 to 10,000, resolves 200 pseudonyms of randomly drawn devices (50 for the scanning strategies above $N = 2{,}000$) and 5 forged ones, counts every H_2 call, and asserts the closed forms of Table 3.2. It writes `measured.csv` and `cost_model.csv`. The OMNeT++ model `ResolveNet` implements the paper's Table II network; its only change to the paper's cost model is that each server's resolution work is read from `cost_model.csv` and priced at $T_H = 0.32$ ms. The ProVerif models are `auth.pv` (secrecy and agreement), `fs.pv` (forward secrecy), and `unlink_counter.pv` and `unlink_clock.pv` (unlinkability). A GitHub Actions workflow runs the test suite with the coverage gate on Python 3.11 and 3.12 and the C vector check on every push and pull request.

4.3 Results of Implementation

Test suite

Table 4.1 lists the test suite. All 125 tests pass, and statement coverage of the protocol and crypto packages is 96% in our re-run (the CI gate is 90%). The property test checks key agreement (Eq. 20) over 10,000 trials, and the adversarial tests cover replay, tampering, forgery, block-listing, a pseudonym-unlinkability smoke test, session-key freshness and the coverage gate.

Table 4.1: Test suite (re-run on 3 Nov 2026: 125 passed)

Test file	**Tests**	**What it checks**
`unit/test_crypto.py`	24	Primitives: hashes, HKDF, AES-GCM, nonces, curve
`unit/test_reference_vectors.py`	15	Python implementation against the independent vectors
`unit/test_resolution.py`	12	Each wire-compatible resolver through the full handshake
`unit/test_device_server.py`	11	End-to-end device–server exchange
`unit/test_ta.py`	10	Enrolment and peer-key sharing
`adversarial/test_sec_adversarial.py`	9	Replay, tampering, forgery, block-list, unlinkability smoke test, key freshness, coverage gate
`unit/test_entity.py`	8	Pairwise-key symmetry, caching, invalidation
`unit/test_session_log.py`	8	Epoch window, replay detection, block-list
`unit/test_messages.py`	7	Wire format of M_1 and M_2
`unit/test_login_gate.py`	6	Optional Section II login pre-check
`unit/test_transport.py`	5	TCP framing
`unit/test_keystore.py`, `unit/test_ta_cli.py`	3 + 3	Key store and enrolment CLI
`property/test_session_key_agreement.py`	2	$\Pr[K_{dev} = K_{srv}] = 1$ (Eq. 20)
`unit/test_demo_integration.py`, `unit/test_rekey_stub.py`	1 + 1	Demo integration; re-key stub
Total	**125**	

Conformance and interoperability

The C device core reproduces all 15 reference-vector checks (Listing 4.3); the Python implementation reproduces the same 15 checks in `test_reference_vectors.py`. In the interoperability run (Listing 4.4) the C core completed five handshakes with the Python server, the session keys agreed in all five, and the server rejected the replayed M_1.

```
ok    epoch0: pairwise root k*
ok    epoch0: M1 wire bytes (PID, C1, mu_1)
ok    epoch0: epoch advanced after M1
ok    epoch0: M2 accepted, session key K
ok    epoch0: tampered M2 ciphertext rejected
ok    epoch0: tampered M2 tag rejected
ok    epoch0: tampered M2 IV rejected
ok    epoch3: pairwise root k*          ... (same seven checks at epoch 3)
ok    replay: M2 of another session rejected
15 checks, 0 failed
```

Listing 4.3: C device core against the reference vectors (`make -C implementation/c/host check`, abridged)

```
RESULT ok epoch=0 m1_bytes=216 fp=ce1fc453f175529e
RESULT ok epoch=1 m1_bytes=216 fp=64a16244598cb0c8
RESULT ok epoch=2 m1_bytes=216 fp=711572f764f5cbd1
RESULT ok epoch=3 m1_bytes=216 fp=f477f3c76b98bfc6
RESULT ok epoch=4 m1_bytes=216 fp=9f478232c437650c
RESULT replay rc=-1 ReplayDetected: pseudonym for 'esp8266-001' reuses stale epoch 4
RESULT summary ok=5/5
session keys agree for 5/5 handshakes: True
server rejected replay: ('rejected', 'ReplayDetected')
```

Listing 4.4: C device core against the Python server (`python implementation/c/tools/interop.py -n 5`)

Command-line demonstration

Listing 4.5 shows the demonstration: enrolment by the trusted authority, one successful authentication, and a replay of the same M_1, which the server rejects. The session-key fingerprint and the latencies differ on every run.

```
$ hea-ta enroll --manifest manifest.json --out-dir creds
Enrolled 'server-A' (role=server) -> creds/server-A.json
Enrolled 'device-A' (role=device) -> creds/device-A.json
2 entities enrolled and cross-issued peer keys. Offline phase complete -- no network I/O occurred.

$ hea-server start --credential creds/server-A.json --host 127.0.0.1 --port 18443

$ hea-device authenticate --credential creds/device-A.json --server-id server-A --server-addr
    127.0.0.1:18443
Mutual authentication succeeded with 'server-A'.
  session key fingerprint: e74617a2626ea74f (SHA-256[:16] of K_ij,t -- never the raw key)
  round-trip latency: 2.103 ms

$ hea-device authenticate ... --replay-last
[replay-demo] resending previously captured M1 (216 bytes) to 127.0.0.1:18443 ...
REJECTED by server: ReplayDetected: pseudonym for 'device-A' reuses stale epoch 0 (round-trip 1.402
     ms)
[replay-demo] rejection confirms the server's freshness check is working as intended.
```

Listing 4.5: Command-line demonstration (one machine, three terminals)

Probe counts at one server

Table 4.2 gives the measured probes per message at one server. The closed-form checks of the measurement script report: for O1, O2, O3, O4 and CR, the forged, per-epoch and state values equal the closed forms exactly at every n; O2 and O4 need exactly 1 and 0 probes per authentication; the honest means of O1, O3 and CR lie within four standard errors of the formulas; MW never exceeds δd on honest and δ on forged input. In the sweep, the TM tables missed 9 of 2,100 honest pseudonyms (the device then retries with its next epoch).

Table 4.2: Measured H_2 probes per message at one server with n enrolled devices (`measured.csv`)

	Honest authentication					**Forged pseudonym**				
n	1	10	100	1,000	10,000	1	10	100	1,000	10,000
O1	18	129.5	1,332	12,846	123,635	25	250	2,500	25,000	250,000
O2	1	1	1	1	1	0	0	0	0	0
O3	1	14.3	161.1	1,576	14,530	3	30	300	3,000	30,000
O4	0	0	0	0	0	0	0	0	0	0
TM	65.6	75.7	271.1	1,478	8,568	98	457.2	2,097	9,634	45,839
CR	1	5.4	49.3	531.0	5,749	1	10	100	1,000	10,000
MW	1	5.3	19.5	33.1	38.6	1	10	4	32	10

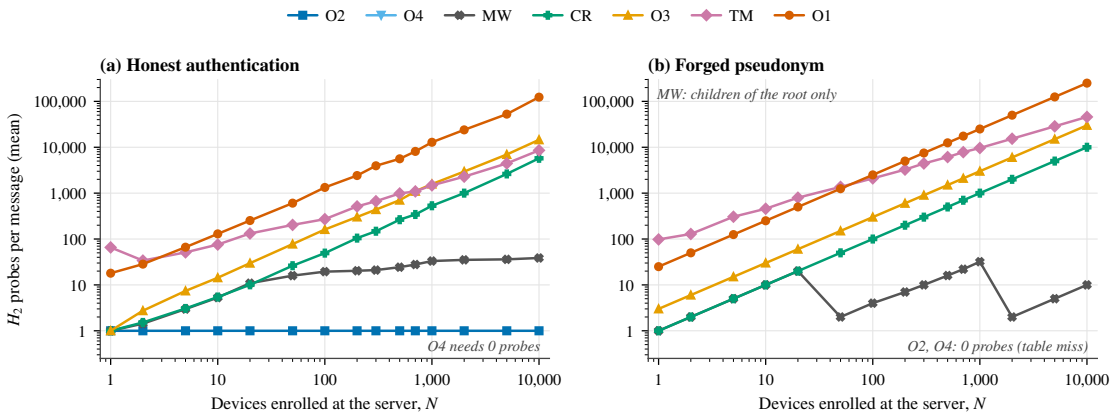


Figure 4.1: Probes per message at one server, measured on the implementation (log–log): (a) honest authentication, (b) a forged, random pseudonym

Session latency and storage in the network model

Table 4.3 ranks the strategies by mean session latency at $N = 1000$ with 5% adversaries, from the 960 OMNeT++ runs; their closed-form costs are given in Table 3.2. The *base* column ($N = 1$) is the value published in [1]: 23.68 ms (Tables III and V) and 320 bits

(Table VI). Storage is Table VI's 320 bits per enrolled device plus the resolution state, averaged over the servers. Figures 4.2 and 4.3 plot the same data.

Table 4.3: Strategies ranked by mean session latency at $N = 1000$ (5% adversaries); mean of five seeds

		Mean session latency (ms)					Storage per server (KB)				
#	**Strategy**	base	10	100	500	1000	base	10	100	500	1000
1	O2 Counter + precomputed table	23.68	24.17	24.25	24.40	24.40	0.04	0.93	8.78	46.1	92.9
2	O4 Clock + per-epoch table	23.68	23.85	23.93	24.22	24.59	0.04	0.47	4.39	23.1	46.4
3	MW Key tree	23.68	24.44	27.96	30.86	31.38	0.04	0.10	0.97	5.16	10.4
4	CR Randomized hash lock	23.68	24.44	28.86	51.71	94.89	0.04	0.10	0.95	5.02	10.1
5	O3 Clock + linear scan	23.68	24.94	38.99	176.1	1,108	0.04	0.16	1.53	8.03	16.1
6	TM Rainbow time–memory trade-off	23.68	38.33	516.5	5,228	12,850	0.04	2.26	10.8	36.5	59.9
7	O1 Counter + linear scan	23.68	36.02	507.9	16,947	44,010	0.04	0.12	1.15	6.02	12.1

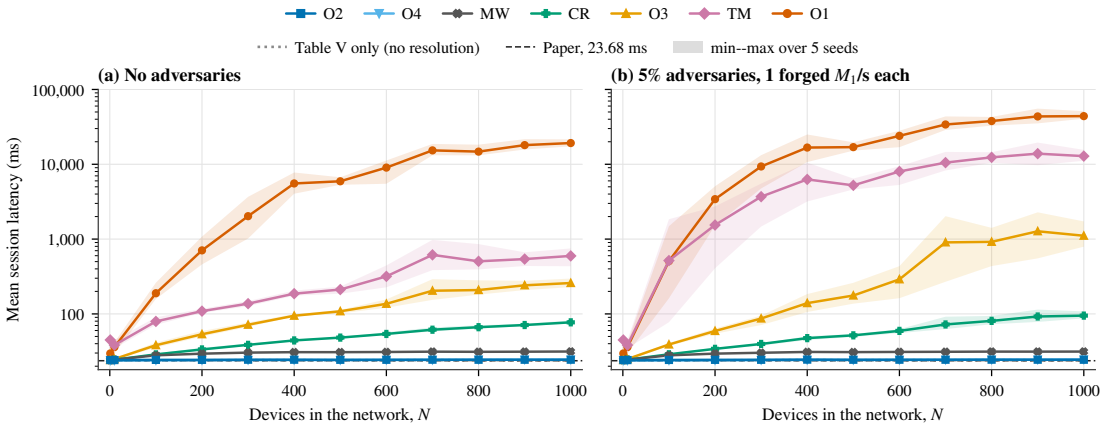


Figure 4.2: Mean session latency against network size in the OMNeT++ model (log scale; band: minimum to maximum over five seeds). Dotted: the paper's cost model with no resolution. Dashed: the paper's 23.68 ms. (a) No adversaries, (b) 5% adversaries

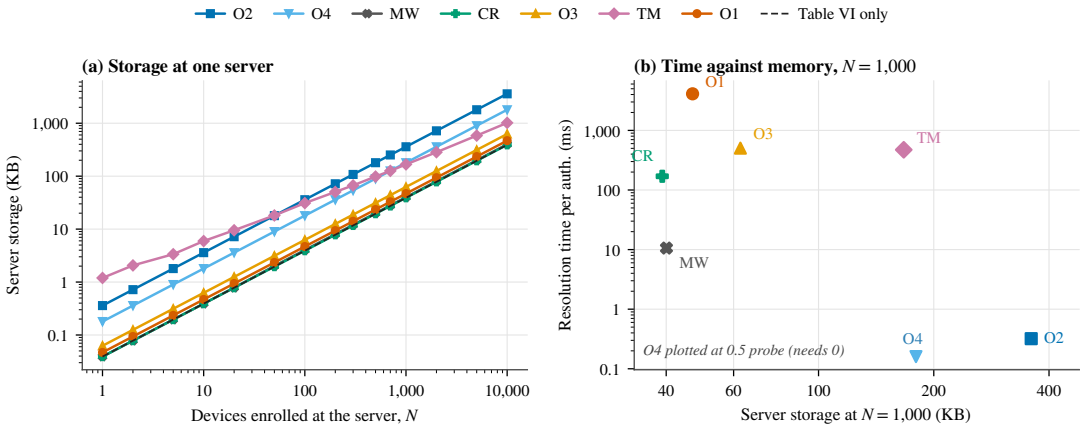


Figure 4.3: (a) Server storage at one server (Table VI's 320 bits per device plus the resolution state); (b) resolution time per authentication ($T_H = 0.32$ ms per probe) against storage at $N = 1000$

Desynchronisation

Table 4.4 is the output of `desync_check.py` for counter epochs (re-run for this report; identical to the committed output). An adversary drops k consecutive M_1 messages, and the next three honest handshakes are attempted.

Table 4.4: Next three honest handshakes after k dropped M_1 messages (counter epochs, $F = 8$)

Dropped k	**Earlier successful session**	**Next three honest handshakes**
6	no / yes	ok, ok, ok
7	no / yes	ok, ok, ok
8	no / yes	UnknownPeerError (all three)
9	no / yes	UnknownPeerError (all three)
12	no / yes	UnknownPeerError (all three)
Device restart after three successful handshakes (epoch kept in RAM): ReplayDetected		

Formal verification

Table 4.5 lists the ProVerif 2.05 results (committed logs in `evaluation/formal_verification/logs/`); every model finishes in under one second.

Table 4.5: ProVerif results

#	**Property**	**Result**
Q1	Session key secret (device side), honest pair	true
Q2	Session key secret (server side), honest device	true
Q3	Server accepts M_1 of honest $V \Rightarrow V$ sent it (non-injective agreement on V, R, N_i, t)	true
Q4	Same, injective (no replay state in the model)	false (replay trace)
Q5	Device accepts $M_2 \Rightarrow$ server derived the same key with the same nonces (injective)	true
FS	Session key secret after a later leak of $k^\star$	false
U1	Unlinkability, counter epochs (observational equivalence)	true
U2	Unlinkability, clock epochs, two sessions in one epoch	fails (equal PIDs)

4.4 Result Analysis

Ranking. The two table strategies, O2 and O4, stay within 1 ms of the paper's 23.68 ms up to 1,000 devices (24.40 ms and 24.59 ms). O2 needs one probe to slide its window and O4 only its per-epoch extension. Their cost is memory: at 1,000 devices about

93 KB and 46 KB per server, against 10 KB for Table VI alone. The key tree (MW) is third at 31.4 ms, but it changes M_1, and a compromised device leaks the internal keys it shares with others. The randomized hash lock (CR) needs no extra state, but its latency grows linearly, to 95 ms. The three scanning strategies over epochs saturate the server: clock scan (O3) at 1.1 s, the time–memory trade-off at 12.9 s, and the counter scan (O1) at 44 s. For O1 and TM, the spread across seeds at $N = 1000$ is 10.6 s and 4.5 s, because latency depends on how many devices and adversaries share the busiest server.

Honest load against forged load. Without adversaries (Figure 4.2a), O1 already reaches 19.2 s at 1,000 devices. Each honest authentication costs about $12.5\,n$ probes, which is 1.0 s of hashing at the $n \approx 258$ devices that a server enrols at $N = 1000$, and arrivals outpace this, so the server queue grows. Forged load (Figure 4.2b) roughly doubles the latency, because every forged pseudonym costs a full miss scan of Wn probes. Such a pseudonym never resolves to a device, so the block-list, which counts failures per device, never sees it. At one server (Figure 4.1b), a forgery at $n = 10{,}000$ costs O1 250,000 probes and TM about 46,000, but costs O2 and O4 nothing, because a table miss needs no hash.

Time against memory. No strategy that keeps the wire format is both fast and small (Figure 4.3b). O2 buys constant time with F table entries per device (328 KB in total at $n = 1000$), and O4 with W_t entries per device (144 KB). TM sits in between, at 131 KB and 1,478 probes, and is dominated here because its table must cover a device's whole lifetime ($m = 128$ epochs), not only the current window. MW is the only point that is both small and fast, and it pays with a different M_1 and a weaker privacy model.

Desynchronisation. With counter epochs the device advances t every time it builds M_1, and the server searches at most F epochs ahead of the last one it accepted. An adversary who drops $F = 8$ consecutive M_1 messages therefore locks the device out for good (Table 4.4), with or without an earlier successful session. The lockout holds for O1 and O2, and a unit test checks it for both. A device whose counter lives in RAM is also rejected as a replay after a restart. Message dropping is within the threat model of [1]. Clock epochs (O3, O4), CR and MW have no per-device counter and cannot desynchronise this way; TM tolerates any number of drops until t reaches the lifetime m.

Security properties. The ProVerif results separate the properties that the resolver cannot affect from those it decides. Session-key secrecy (Q1, Q2), agreement on (V, R, N_i, t) (Q3) and injective key agreement for the device (Q5) hold for every correct resolver; Q5 holds because N_i is fresh and bound as associated data of M_2. Injective authentication of the device (Q4) fails without replay state: ProVerif finds a trace in which one M_1 is accepted twice, so injectivity rests on the epoch window (O1, O2, TM) or the nonce cache

(O3, O4), which the specification leaves open. With clock epochs the attacker links two sessions in one epoch by testing their pseudonyms for equality (U2); with counter epochs the equivalence holds (U1). Forward secrecy does not hold, and the base paper does not claim it: a later leak of $k^\star$ reveals N_i, N_j and t from the ciphertexts and with them the session key.

Comparison with the objectives. Table 4.6 relates each objective of Section 1.3 to the evidence obtained.

Table 4.6: Objectives and the evidence obtained

#	**Objective**	**Evidence**
1	Conformant Python and C implementations	15/15 vector checks in each; 5/5 interoperable handshakes; 125 tests pass
2	Resolution required; open choices identified	Five-step argument from the specification and the vectors (Section 3.3); two open choices
3	Seven strategies, measured and validated	Closed forms exact for O1–O4 and CR; honest means within 4 SE; MW within its bound; TM 9 misses in 2,100
4	Network pricing and ranking	960 OMNeT++ runs; O2 at 24.4 ms and O1 at 44 s at $N = 1000$ with 5% adversaries
5	Failure modes	Forged pseudonyms cost O1 *Wn* probes and O2/O4 none; lockout after 8 dropped messages
6	Formal analysis	Q1–Q3, Q5, U1 true; Q4, FS false; U2 fails
7	Open release	Code, models, data and logs in the public repository

4.5 Limitations

- The network model is our model of the paper's stated environment, not the authors' simulation, which is not available. The adversary share and rate, the 10 s authentication window and the enrolment rule are our assumptions, and the saturated latencies of O1, O3 and TM depend on them.
- Probes are priced with the paper's constant $T_H = 0.32$ ms; no hardware timings are reported.
- CR and MW change M_1 and are modelled at the resolution level only, not in the full handshake.
- The TM parameters (six tables, $m = 128$) are one point on its trade-off curve, and TM measurements vary between runs because peer secrets are drawn from operating-system randomness.
- The epoch, window and table parameters are our choices, not the paper's; confor-

mance means agreement with our reading of Section III.

- The Python key store is demonstration-grade, re-keying (Eq. 24) is a stub, and coverage (Eqs. 10, 23) is an abstract predicate with no radio model.

4.6 Observation/Remarks

- **The original implementation was the worst case.** Our first server, written before the resolution study, resolved pseudonyms with a linear scan over a counter window, which is strategy O1. Nothing in the paper warned against it, and it turned out to be the slowest strategy and the one most exposed to forged traffic.
- **A precomputed table removes both the scaling and the denial-of-service problem.** Because a table miss needs no hash, forged pseudonyms become free for the server to reject, at a cost of about $40F$ bytes per device.
- **The block-list does not protect resolution.** It counts failures per device, but a forged pseudonym never resolves to a device, so the expensive work happens before the block-list can act.
- **Specification ambiguities.** Section II and Section III of the paper describe different online phases; the implementation follows Section III, the version covered by the paper's security sketch. The paper also leaves open whether μ_1 covers the clear pseudonym and how σ_2 is built; these choices are recorded in the assumption register.
- **Reproducibility of the measurements.** Re-running the probe sweep for this report reproduced every non-TM row of `measured.csv` exactly. The TM rows differ, as the repository documents: in our re-run the honest probes per $S^{2/3}$ ranged from 0.42 to 0.67 (0.50 to 0.73 in the committed run) and 7 of 2,100 pseudonyms were missed (9 in the committed run). Regenerating the figures and the ranked table from the committed data reproduced them exactly.
- **Independent vectors were essential.** Generating the vectors with code that shares nothing with either implementation fixed the message layout on which the whole gap argument relies, and made the C port's conformance checkable in CI without hardware.

4.7 Conclusion

The protocol of [1] was implemented in Python and, on the device side, in C, and both reproduce the independent reference vectors byte for byte. Seven resolution strategies were implemented and measured, and their probe counts match the closed forms. Under the paper's own constants and network, the choice of resolver decides whether the server

stays at the paper's 23.68 ms per session or saturates at tens of seconds: O2 holds 24.4 ms at 1,000 devices with 5% adversaries, while O1 reaches 44 s. The choice also decides whether forged pseudonyms are free to send and whether eight dropped messages lock a device out. The formal analysis shows that secrecy and agreement do not depend on the choice, while unlinkability and injective authentication do. All seven objectives were met, within the limitations listed above. The next chapter concludes the report and outlines future work.

CHAPTER: 5

CONCLUSION

Chapter 5

Conclusion

5.1 Limitations

- **Model, not the authors' simulation.** The OMNeT++ network is our model of the environment stated in Table II of [1]; the authors' simulation is not available. The adversary rate, the authentication window and the enrolment rule are our assumptions, and the saturated latencies of the scanning strategies depend on them.
- **No hardware timings.** Every probe is priced with the paper's constant $T_H = 0.32\,\text{ms}$. The ESP8266 port demonstrates that the device side runs on real hardware, but the reported latencies are not measured on it.
- **Partial modelling of two strategies.** CR and MW change the first message and are modelled at the resolution level only.
- **One point on each trade-off.** The TM parameters (six tables, $m = 128$) and the window parameters ($L = 16$, $F = 8$, $W_t = 3$) are single choices; other choices move each strategy along its curve.
- **Our reading of the specification.** Epoch semantics, windows and tables are our choices, and conformance means agreement with our reading of Section III, which only the authors can confirm.
- **Symbolic verification only.** The ProVerif results assume perfect primitives and do not include the server's replay state, so injective device authentication (Q4) is checked by test rather than proved.
- **Prototype scope.** Devices are static, coverage is an abstract predicate, re-keying is a stub, the key store is demonstration-grade, and side channels are out of scope.

5.2 Conclusion

A server cannot authenticate a device that it has not yet identified. In the two-message protocol of Al-Rasheed et al. [1], that identification, pseudonym resolution, is a required step with no specification and no cost in the model. This project showed from the specification alone that every correct server must perform it, implemented the protocol in Python and on the device side in C with byte-for-byte conformance to independent vectors, and compared seven ways of filling the gap under the paper's own constants and

network.

The choice turned out to decide the protocol's practical behaviour. With a precomputed table (O2) the server stays within 1 ms of the published 23.68 ms per session at 1,000 devices and rejects forged pseudonyms for free, at a memory cost of tens of kilobytes per server. With the on-demand counter scan that the specification suggests (O1), the session latency reaches about 44 s at the same scale with 5% adversaries, forged pseudonyms are an effective denial-of-service attack that the block-list never sees, and an attacker who drops eight messages locks a device out for good. Clock epochs avoid the lockout but make sessions in one epoch linkable. The formal analysis shows that the protocol's secrecy and agreement properties do not depend on the choice, whereas unlinkability and injective authentication do, and that the protocol has no forward secrecy.

All seven objectives of the project were achieved. The practical recommendation is that a deployment with counter epochs should precompute (O2) and add a resynchronisation path, that a deployment with clocks can use O4 if it accepts linkability within an epoch and keeps a nonce cache, and that the specification should state the epoch semantics, the resolution method and the replay state, and add a resolution term to its cost model. Precomputed tables, the oldest answer from RFID private identification, keep the protocol's cost claim intact.

5.3 Future Scope

- **Stateful verification.** Prove injective device authentication with the replay window included, for example in Tamarin, which supports mutable state.
- **Hardware measurements.** Measure resolution and handshake times on the ESP8266 and on a server-class machine, and replace the paper's T_H constant with measured values.
- **Resynchronisation.** Design and verify a recovery path for counter epochs after the lockout, and study the trade-off between a larger forward window F and table size.
- **Full handshakes for CR and MW.** Integrate the strategies that change M_1 into the full exchange and the simulator, and quantify MW's privacy loss under device compromise.
- **Broader parameter sweeps.** Explore the TM trade-off curve, epoch lengths for O4, adversary rates, mobility and hand-off.
- **Re-keying and forward secrecy.** Implement the TA-assisted re-keying of Eq. 24 and evaluate extensions that add forward secrecy.
- **Other protocols.** Apply the same method, i.e. derive the resolution step from the

specification, measure it, and price it under the paper's own constants, to other pseudonym-based IoT authentication schemes.

References

[1] A. Al-Rasheed, R. Khan, A. Ahmad, A. Saeed, F. Alturise and S. Alkhalaf, "A low-cost hybrid elliptic curve cryptography authentication protocol for trustworthy Internet of Things communication," *IEEE Trans. Consum. Electron.*, vol. 72, no. 2, pp. 4483–4491, May 2026, doi:10.1109/TCE.2026.3662053.

[2] S. A. Weis, S. E. Sarma, R. L. Rivest and D. W. Engels, "Security and privacy aspects of low-cost radio frequency identification systems," in *Security in Pervasive Computing (SPC 2003)*, LNCS 2802. Springer, 2004, pp. 201–212.

[3] M. Ohkubo, K. Suzuki and S. Kinoshita, "Cryptographic approach to 'privacy-friendly' tags," in *RFID Privacy Workshop*, MIT, Cambridge, MA, USA, Nov. 2003.

[4] D. Molnar and D. Wagner, "Privacy and security in library RFID: Issues, practices, and architectures," in *Proc. 11th ACM Conf. Computer and Communications Security (CCS)*, 2004, pp. 210–219.

[5] G. Avoine and P. Oechslin, "A scalable and provably secure hash-based RFID protocol," in *Proc. 3rd IEEE Int. Conf. Pervasive Computing and Communications Workshops (PerCom Workshops)*, 2005, pp. 110–114.

[6] G. Avoine, E. Dysli and P. Oechslin, "Reducing time complexity in RFID systems," in *Selected Areas in Cryptography (SAC 2005)*, LNCS 3897. Springer, 2006, pp. 291–306.

[7] G. Tsudik, "YA-TRAP: Yet another trivial RFID authentication protocol," in *Proc. 4th IEEE Int. Conf. Pervasive Computing and Communications Workshops (PerCom Workshops)*, 2006, pp. 640–643.

[8] M. E. Hellman, "A cryptanalytic time-memory trade-off," *IEEE Trans. Inf. Theory*, vol. 26, no. 4, pp. 401–406, 1980.

[9] P. Oechslin, "Making a faster cryptanalytic time-memory trade-off," in *Advances in Cryptology – CRYPTO 2003*, LNCS 2729. Springer, 2003, pp. 617–630.

[10] A. Varga and R. Hornig, "An overview of the OMNeT++ simulation environment," in *Proc. SIMUTools*, 2008.

[11] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, 2010.

[12] National Institute of Standards and Technology, “Secure hash standard (SHS),” FIPS PUB 180-4, 2015.

[13] M. Dworkin, “Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM),” NIST SP 800-38D, 2007.

[14] B. Blanchet, “Modeling and verifying security protocols with the applied pi calculus and ProVerif,” *Foundations and Trends in Privacy and Security*, vol. 1, no. 1–2, pp. 1–135, 2016.

[15] B. Blanchet, M. Abadi and C. Fournet, “Automated verification of selected equivalences for security protocols,” *J. Log. Algebr. Program.*, vol. 75, no. 1, pp. 3–51, 2008.

[16] M. Arapinis, T. Chothia, E. Ritter and M. Ryan, “Analysing unlinkability and anonymity using the applied pi calculus,” in *Proc. 23rd IEEE Computer Security Foundations Symp. (CSF)*, 2010, pp. 107–121.

[17] J. Kerkar, C. Kelkar, C. Poornamath, Y. Kolekar and S. Sondur, “hybrid-ecc-iot-auth: Implementation, models and data for pseudonym resolution in two-message IoT authentication,” 2026. [Online]. Available: `https://github.com/jaykerkar0405/hybrid-ecc-iot-auth`

Appendix

A.I. Publication Details

Paper: J. Kerkar, C. Kelkar, C. Poornamath, Y. Kolekar and S. Sondur, "Pseudonym Resolution and Formal Verification of an Unlinkable Two-Message IoT Authentication Protocol," 2026. Department of Information Technology, Vivekanand Education Society's Institute of Technology, Mumbai. Source and PDF: `paper/` in the project repository [17].

Status: Unpublished manuscript (as recorded in the repository). Venue, submission and acceptance details: **[FLAG F12]**

Camera Ready / Published Paper

[FLAG F12] Attach the camera-ready or published version here (the current manuscript is `paper/main.pdf`).

Plagiarism report

[FLAG F13] Attach the plagiarism report required by the department.

A.II. IPR: Copyright/Patent/etc. (if granted)

The software is released under the Apache License 2.0, copyright 2026 Jay Kerkar, Chaitanya Kelkar, Chirag Poornamath, Yash Kolekar and Shanta Sondur. No registered copyright or patent is recorded in the repository. **[FLAG F14]**